

怎麼用postman測試https網址，外部是否能存取

已思考 15s ▾

用 Postman 測 https 網址，其實就是「從你電腦 → 網路 → 目標網站」發一個 HTTPS 要求，看能不能拿到回應。

要確認「外部是否能存取」，重點是：你要從真正的外網環境測，不要在同一台主機 / 內網測。

我分成兩個部分講：

一、基本 Postman 測試步驟

1. 開啟 Postman，新建一個 Request

- 左上角 + 新增 Tab 或 New → HTTP Request。

2. 選 Method + 貼網址

- 通常先選 GET。
- 在 URL 欄貼上你的 https 網址，例如：
 - `https://example.com`
 - `https://your-domain.com/api/v1/test`
- 確定是「外部可以解析到的」網域或 IP：
 - ❌ `localhost`、`127.0.0.1`、`192.168.x.x`、`10.x.x.x` 這些都是內網
 - ✅ 公網 IP 或 DNS 網域名

3. 按 Send 送出請求

- 右上角 Send。

4. 看 Response 結果判斷

如果外部「可以」存取，大致會出現：

- 有 status code：`200` / `301` / `404` / `500` ... 都代表：

Postman 有連到你的服務，伺服器有回應。

- 下方會有 Response body、Headers 等。
- 這種情況就表示：至少從你現在這個網路，是可以連到該 https 網址的。

如果外部「無法」存取，常見情況是：

- Postman 顯示：

- `Could not get any response`
- `ECONNREFUSED` (連線被拒絕，可能沒開服務 / 防火牆擋)
- `ETIMEDOUT` (連線逾時，多半是 IP/Port 沒打通或被擋)
- `ENOTFOUND` (DNS 找不到網域，代表網域沒設定好或未生效)
- 這些都表示：從你現在這個網路，連不到那個網址。

二、如何確認「外部」真的能存取？

很多人會在 伺服器本機 或 內網電腦 用 Postman 測，
結果成功了，但這只是「內部可以」，不代表「外部可以」。

你要檢查的是：

1. 從不同網路測一次

- 例如：
 - 用你自己電腦 + 手機 4G 熱點上網再開 Postman 測一次。
 - 或換另一個 Wi-Fi。
- 如果：
 - 內網可以、4G 不行 → 很可能是防火牆 / Security Group / Port 沒開對外。
 - 兩邊都可以 → 代表外部大致上沒問題 (至少這兩個網路測 OK)。

2. 確認 URL 是對外公開的位址

- 如果你用的是 `https://localhost:5000`、`https://192.168.1.100:443` 這種：
 - 這只能在內網或本機測，外面的人用不到。
- 要讓外面的人存取，需要：
 - 公網 IP / 公開網域
 - Server / 防火牆安全性設定允許該 Port 對外 (例如 443)。

3. SSL 憑證相關 (如果是自簽憑證或測試憑證)

- 如果你的網站用自簽 SSL，Postman 可能會跳 SSL 錯誤，如：
 - `SSL Error: self signed certificate`
- 想先單純測連線，可在 Postman 暫時關掉 SSL 檢查：
 - `File → Settings → General → SSL certificate verification` 選 Off
(或右上角齒輪 → Settings，看版本)
- 關掉後再測一次：
 - 如果這樣就能拿到 Response，代表「網路是通的，只是憑證不被信任」。

三、進階：寫一個簡單的測試腳本 (非必要)

在 Postman 的 **Tests** 分頁，可以寫簡單檢查，例如：

javascript

複製程式碼

```
pm.test("Status code is 200", function () {  
    pm.response.to.have.status(200);  
});
```

- 如果狀態碼不是 200，右邊 Tests 就會顯示 Fail，適合之後自動化測試用。

四、簡單總結給你一個判斷邏輯

1. 用 Postman 對 https 網址送請求 (GET 就好) 。
2. 看結果：
 - 有任何 HTTP status code (200/3xx/4xx/5xx) → 表示連得到那台機器。
 - 完全沒回應 / timeout / DNS not found → 表示外部「連不到」。
3. 再從不同網路 (例如手機 4G) 重複一次：
 - A 網路可以、B 網路不行 → 可能是某些網路被擋 / 路由 / 防火牆問題。
 - 都不行 → 網站本身沒對外開好，或 DNS/Port 沒設通。

如果你願意的話，可以把你要測的 **大概網址格式** (遮掉 domain 也沒關係，例如：

`https://xxx.com/api/login`) 跟目前看到的 **Postman 錯誤訊息** 直接貼上來，我可以幫你一起判斷是哪一種情況。

📄 🍌 🗨 📌 🔄 ...

```
{  
  "errorMessage": "httpMethod",  
  "errorType": "KeyError",  
  "requestId": "e15390ed-2115-4f08-9a04-  
6c3641d9e890",  
  "stackTrace": [  
    " File \"/var/task/lambda_function.py", line 6, in  
lambda_handler\n  return awsgi.response(app, event,  
context)\n",  
    " File \"/var/task/awsgi/__init__.py", line 172, in  
response\n  output = app(envron(event, context),  
sr)\n",
```

```
" File \"/var/task/awsgi/__init__.py\", line 121, in
environ\n 'REQUEST_METHOD':
event['httpMethod'],\n"
]
}postman 回傳
```

已思考 7s >

你這個錯誤其實不是 Postman 的問題，是 **Lambda** 收到的 **event** 格式跟 **awsgi** 預期的不一樣。

一、錯誤在說什麼？

錯誤訊息重點：

json

 複製程式碼

```
"errorMessage": "'httpMethod'",
"errorType": "KeyError",
...
" File \"/var/task/awsgi/__init__.py\", line 121, in environ
  'REQUEST_METHOD': event['httpMethod'],
```

意思是：

awsgi 在組 Flask 的 **environ** 時，去讀 `event['httpMethod']`，
結果你的 **event** 裡根本沒有 **httpMethod** 這個欄位，所以 **KeyError**。

這通常發生在：

- 你用的是 **Lambda Function URL** 或 **HTTP API (API Gateway v2)**
- 但 **awsgi** 這個版本是依照 **舊的 REST API (proxy integration)** 的 **event** 格式寫的
- 舊版 REST API 事件裡有 **httpMethod**，新版的沒有

二、不同觸發方式，event 長得不一樣

1. REST API (API Gateway v1, Lambda Proxy Integration)

event 大概長這樣（**awsgi** 預期的）：

json

 複製程式碼

```
{
  "resource": "/{proxy+}",
```

```
"path": "/hello",
"httpMethod": "GET",
"headers": { ... },
"queryStringParameters": { ... },
...
}
```

這裡就有 `httpMethod` 。

2. HTTP API / Lambda Function URL

event 會長這樣：

json

 複製程式碼

```
{
  "version": "2.0",
  "routeKey": "$default",
  "rawPath": "/",
  "rawQueryString": "",
  "requestContext": {
    "http": {
      "method": "GET",
      "path": "/",
      ...
    }
  },
  "headers": { ... },
  "queryStringParameters": { ... }
}
```

這種格式 沒有 `event['httpMethod']` 。

HTTP 方法藏在：`event["requestContext"]["http"]["method"]` 。

所以 `awsgi` 說：找不到 `httpMethod` → `KeyError` 。

三、你現在用 Postman 測試 https，其實已經「打到 Lambda」，只是 Lambda wrapper 不吃

你現在流程大概是：

Postman → HTTPS (API Gateway or Function URL) → Lambda → `lambda_handler` → `awsgi.response(app, event, context)` → 這邊爆。

這代表：

- 網路、HTTPS、Postman 那邊是「通的」
- 問題在於：**Lambda handler + awsgi** 跟觸發方式不相容

四、你可以怎麼修？

有幾種路可以選，你挑一種即可：

方案 A：改用 REST API (API Gateway v1 + Lambda Proxy Integration)

如果你是跟著某些舊教學（例如：用 awsgi 跑 Flask + Lambda），

很多都是假設你用的是 **REST API + Lambda Proxy**。

你可以：

1. 在 API Gateway 建立 REST API
2. 方式選 **Lambda Proxy Integration**
3. 指向你的 Lambda
4. Deploy API，拿到 `https://xxxx.execute-api.region.amazonaws.com/prod/...`
5. 用 Postman 打這個 URL

這樣 event 會有 `httpMethod`，就配得起 awsgi 的預設。

方案 B：繼續用 Lambda Function URL / HTTP API，但換 Handler / Middleware

如果你想用 **Lambda Function URL** 或 **HTTP API (v2)**，那就有兩條路：

B-1：不用 awsgi，自己解析 event

直接在 `lambda_function.py` 裡寫類似：

python

 複製程式碼

```
def lambda_handler(event, context):
    method = event["requestContext"]["http"]["method"]
    path = event["rawPath"]
    # 根據 method + path 自己路由，回傳 dict
    return {
        "statusCode": 200,
        "headers": {"Content-Type": "application/json"},
```

```
"body": json.dumps({"message": "ok"})
}
```

就變成「原生 Lambda HTTP 處理」，不用 Flask/awsgi。

B-2：換成支援 HTTP API / Function URL 的框架/版本

有一些 library (例如 mangum for FastAPI/Starlette，或 awsgi 新版/其他 fork)

支援 API Gateway v2 / Lambda URL，不過你要看：

- 你現在用的 awsgi 是哪個版本？
- 它有沒有文件說明支援 HTTP API / v2 event？

如果你真的要用 Flask + Lambda Function URL，比較常見的是用 **serverless-wsgi** 或其他 wrapper，看它是否有 `event["requestContext"]["http"]["method"]` 的支援。

方案 C：自己在 awsgi 外面做一層「event 轉換」

如果你不想換架構，也可以在 `lambda_handler` 先把 event 轉成 awsgi 喜歡的格式：

python

複製程式碼

```
import awsgi
from app import app # 你的 Flask app

def lambda_handler(event, context):
    # 如果是 HTTP API / Function URL v2 event，就包一層
    if "requestContext" in event and "http" in event["requestContext"]:
        method = event["requestContext"]["http"]["method"]
        path = event.get("rawPath", "/")
        # 做一個假的 v1 event
        event_v1 = {
            "path": path,
            "httpMethod": method,
            "headers": event.get("headers", {}),
            "queryStringParameters": event.get("queryStringParameters"),
            "body": event.get("body"),
            "isBase64Encoded": event.get("isBase64Encoded", False)
        }
        return awsgi.response(app, event_v1, context)

    # 否則當作原本 v1 event
    return awsgi.response(app, event, context)
```

這樣就不會再因為 `httpMethod` 不存在而噴錯。

五、跟「外部是否能存取」的結論

從你貼的錯誤可以推論：

- Postman 的 HTTPS 請求 **確實打到 Lambda** 了 (不然不會出現 Lambda 的 stackTrace)
- 只是 Lambda 裡面的 awsgi 對 event 結構不滿意

所以：

-  你的 https URL 已經「對外可達」 (至少你這個網路是可以)
-  目前 Lambda 的 Handler 沒有處理好 HTTP API / Function URL 的 event 格式

如果你願意，你可以再貼：

- 你的 `lambda_function.py` 內容
- 你這個 https 是：
 - API Gateway REST API、HTTP API，還是 Lambda Function URL？
- AWS Console 裡 event 測試樣本 (或是你自己設計的 test event JSON)

我可以直接幫你改到可以被 Postman 正常打通。



```
{  
  "error": "data"  
}
```

我有設api 還有選lambda proxy

已思考 30s >

這次看到的：

json

 複製程式碼

```
{  
  "error": "'data'"  
}
```


這就不是 AWS / Postman 的問題了，而是你自己的程式裡在用 `['data']` 這個 key，結果 request 裡沒有這個欄位，所以被你程式包成 `{"error": "'data'"}` 回給你。

換句話說：

✅ Postman 已經成功打到 API Gateway → Lambda → Flask

❌ 但是你的 Flask / Lambda 裡面在取 `data` 這個欄位時噴了 `KeyError('data')`

1. 為什麼會出現 `"'data'"` ？

典型會長這樣：

python

複製程式碼

```
@app.route("/xxx", methods=["POST"])
def xxx():
    body = request.get_json() # 或 request.json
    prompt = body["data"]    # ← 這行，如果 JSON 裡沒有 data，就會 KeyError
```

或 Lambda 裡：

python

複製程式碼

```
event_body = json.loads(event["body"])
something = event_body["data"] # 這裡也一樣
```

如果 Postman 送過去的是：

- GET，根本沒 body
- 或 POST，但 body 是空的 / 沒有 `"data"` 這個欄位
→ 你的程式在 `["data"]` 那行就爆，然後 error handler 把它包成：

json

複製程式碼

```
{"error": "'data'"}
```

2. 先確認：代表外部「可以連到」你的網址

你現在能收到這個 JSON：

json

複製程式碼

```
{"error": "'data'"}
```

代表整個路徑是通的：

- Postman → `https://你的 API 網址` (API Gateway, Lambda Proxy)
- Lambda 有被觸發
- Flask / awsgi 有跑到你的 route
- 只是 route 裡讀 body 的時候炸掉

所以從「外部能否存取」這個角度來說：✅ 可以。

現在要處理的是「怎麼正確呼叫你的 API」。

3. 用 Postman 正確送出帶 `data` 的 JSON

假設你的後端真的就是在讀 `body["data"]`，那在 Postman 要這樣設：

1. Method 選 `POST` (你的 route 如果是 `POST`)。
2. URL 貼上你的 `https` 網址，例如：

```
https://xxxxx.execute-api.ap-northeast-1.amazonaws.com/prod/你的路徑
```

3. Headers 多一個：

- `Content-Type: application/json`

4. Body → 選 `raw` → `JSON`，內容類似：

json

複製程式碼

```
{  
  "data": "這裡放你原本預期的內容"  
}
```

如果你的程式是像這樣：

python

複製程式碼

```
body = request.get_json()  
prompt = body["data"]["prompt"]
```

那你在 Postman 裡就要送：

json

複製程式碼

```
{  
  "data": {
```

```
"prompt": "畫一張可愛的貓"
}
}
```

重點：你的 Postman body 結構，要跟程式裡 `body["data"]...` 的結構對得起來，不然一定噴 error。

4. 程式端建議加上防呆（避免 KeyError）

不然每次少一個欄位就直接 500，很難 debug。

你可以改成：

python

複製程式碼

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route("/xxx", methods=["POST"])
def xxx():
    if not request.is_json:
        return jsonify({"error": "Body must be JSON"}), 400

    body = request.get_json(silent=True) or {}
    # 用 get 而不要用 ["data"]
    data = body.get("data")
    if data is None:
        return jsonify({"error": "missing 'data' field in JSON body"}), 400

    # 後面再對 data 做處理
    # ...
    return jsonify({"ok": True})
```

這樣如果 Postman 沒帶 `data`，你會拿到明確訊息：

json

複製程式碼

```
{
  "error": "missing 'data' field in JSON body"
}
```

而不是一個怪怪的 `''data''`。

如果你是直接在 Lambda 解析 event 的話，有點類似：

python

 複製程式碼

```
def lambda_handler(event, context):
    body = json.loads(event.get("body") or "{}")
    if "data" not in body:
        return {
            "statusCode": 400,
            "headers": {"Content-Type": "application/json"},
            "body": json.dumps({"error": "missing 'data' field"})
        }

    data = body["data"]
    # ...
```

5. 如果你只是想測「健康檢查 / 外部可達」，可以加一個簡單 GET

你可以在 Flask 多寫一個不吃 body 的 route，專門拿來 Postman 測：

python

 複製程式碼

```
@app.route("/health", methods=["GET"])
def health():
    return jsonify({"status": "ok"}), 200
```

然後在 API Gateway 把 `/health` 也 mapping 出去：

- 用 Postman GET `https://你的網址/health`
- 有收到 `{"status": "ok"}` 就代表：外部 → API → Lambda → Flask 全部沒問題。

這樣你就可以分清楚：

- `/health` 成功 → 網路 OK
- `/你的真正 API` 回 `{"error": "'data'"}` → Body 結構沒對

小結

1. `{"error": "'data'"}` 是你的程式在讀 `["data"]` 時炸掉，不是網路問題。
2. 這其實已經證明：Postman 從外部可以打到你的 https API。
3. 接下來要做的是：
 - Postman 依照你程式預期的 JSON 結構送 `data`。

- 或修改程式改成 `get("data")` + 做缺欄位檢查。
- 額外加一個 `/health` GET，方便你測「外部是否能存取」。

如果你願意，可以把你 Flask 裡那個 route（有用到 `["data"]` 的那段）貼上來，我可以直接幫你改成「正確 Postman body 範例 + 防呆版本」。



```
{
  "resource": "/audio-callback",
  "path": "/audio-callback",
  "httpMethod": "POST",
  "headers": {
    "Content-Type": "application/json"
  },
  "multiValueHeaders": {
    "Content-Type": [
      "application/json"
    ]
  },
  "queryStringParameters": null,
  "multiValueQueryStringParameters": null,
  "pathParameters": null,
  "stageVariables": null,
  "requestContext": {
    "path": "/audio-callback",
    "stage": "prod",
    "identity": {
      "sourceIp": "127.0.0.1",
      "userAgent": "lambda-test-event"
    }
  },
  "body": "{\"code\":200,\"data\":\n{\n\"callbackType\":\"\ncomplete\",\"\n\"data\":\n{\n{\n\"audio_url\":\"\nhttps://musicfile.api.box/MjU4ZDA4Yj\nUtNDJmNi00ZDFhLTg1MGMtNDZIY2JiNzg2ZDAz.mp3\n\",\"\n\"createTime\":1763910096539,\"\n\"duration\":107.92,\"\ni\n\"d\":\"\n258d08b5-42f6-4d1a-850c-46ecbb786d03\",\"\ni\n\"image_url\":\"\nhttps://musicfile.api.b\nox/MjU4ZDA4YjUtNDJmNi00ZDFhLTg1MGMtNDZIY2Ji\nNzg2ZDAz.jpeg\",\"\n\"model_name\":\"\nchirp-v3-5\",\"\n\"prompt\":\"\n\",\"\n\"source_audio_url\":\"\nhttps://cdn1.
```

```

suno.ai/258d08b5-42f6-4d1a-850c-
46ecbb786d03.mp3\", \"source_image_url\": \"https://cd
n2.suno.ai/image_258d08b5-42f6-4d1a-850c-
46ecbb786d03.jpeg\", \"source_stream_audio_url\": \"htt
ps://cdn1.suno.ai/258d08b5-42f6-4d1a-850c-
46ecbb786d03.mp3\", \"stream_audio_url\": \"https://m
usicfile.api.box/MjU4ZDA4YjUtNDJmNi00ZDFhLTg1MG
MtNDZIY2JiNzg2ZDAz\", \"tags\": \"古典\", \"title\": \"宁静
钢琴冥想\"},
{ \"audio_url\": \"https://musicfile.api.box/YWVjYzIxYzAt
NDk1MC00MTVmLTg1ODUtODFmNzRjNDkxZGRj.mp3
\", \"createTime\": 1763910096539, \"duration\": 168.36, \"
id\": \"aecc91c0-4950-415f-8585-
81f74c491ddc\", \"image_url\": \"https://musicfile.api.bo
x/YWVjYzIxYzAtNDk1MC00MTVmLTg1ODUtODFmNzR
jNDkxZGRj.jpeg\", \"model_name\": \"chirp-v3-
5\", \"prompt\": \"\", \"source_audio_url\": \"https://cdn1.
suno.ai/aecc91c0-4950-415f-8585-
81f74c491ddc.mp3\", \"source_image_url\": \"https://cd
n2.suno.ai/image_aecc91c0-4950-415f-8585-
81f74c491ddc.jpeg\", \"source_stream_audio_url\": \"htt
ps://cdn1.suno.ai/aecc91c0-4950-415f-8585-
81f74c491ddc.mp3\", \"stream_audio_url\": \"https://mu
sicfile.api.box/YWVjYzIxYzAtNDk1MC00MTVmLTg1OD
UtODFmNzRjNDkxZGRj\", \"tags\": \"古典\", \"title\": \"宁
静钢琴冥想
\"}], \"task_id\": \"2b00f76847c403fe9051b70c95bc27da
\", \"msg\": \"All generated successfully.\"},
  \"isBase64Encoded\": false
}

```

```

@app.route('/audio-callback', methods=['POST'])
def audio_callback():
    print("★ 進到 audio_callback")
    try:
        print("=== 收到 Suno callback ===")
        data = request.json
        suno_items = data["data"]["data"]

        for item in suno_items:
            audio_url = item.get("source_audio_url")
            music_id = item["id"]

```

```
if not audio_url:
    print("❌ 沒有 audio_url")
    return jsonify({
        "success": False,
        "error": "Suno 沒有回傳 audio_url"
    }), 500

# 下載音檔
mp3 = requests.get(audio_url).content

# 上傳 S3
timestamp =
datetime.datetime.now().strftime("%Y%m%d%H%M%S")
filename = f"
{timestamp}_{uuid.uuid4().hex[:8]}.mp3"
s3_key = f"Audios/{filename}"

try:
    s3.put_object(
        Bucket=S3_BUCKET_NAME,
        Key=s3_key,
        Body=mp3,    # 直接用原始 bytes
        ContentType="audio/mpeg"
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"上傳 S3 失敗: {str(e)}"
    }), 500

public_url=f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"
local_url=f"s3://{S3_BUCKET_NAME}/{s3_key}"

# 存進 DynamoDB
try:
    result = insert_record(
        user_id="no user id",
        prompt="no prompt",
        file_url=public_url,
        record_type="audio",
```

```
        status="success"
    )
except Exception as e:
    return jsonify({
        "statusCode": 500,
        "body": json.dumps({
            "message": "✖ 插入失敗",
            "error": str(e)
        })
    })
# 提供可存取的 url
try:
    presigned_url = s3.generate_presigned_url(
        ClientMethod="get_object",
        Params={
            "Bucket": S3_BUCKET_NAME,
            "Key": s3_key
        },
        ExpiresIn=3600 # 有效時間秒數
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"產生下載網址失敗: {str(e)}"
    }), 500

return jsonify({
    "success": True,
    "download_url": presigned_url, # 👉 這是 S3
    "reply": "Flask 收到音樂prompt"
})

except Exception as e:
    print("callback error:", e)
    return jsonify({"error": str(e)}), 500
```

上面的是testevent，下面的是後端內容

已思考 49s >

啊哈，現在問題就清楚了，你這個 `''data''` 其實是 測試資料送錯東西，不是後端邏輯本身壞掉。

你貼的有兩個東西：

1. Lambda 測試事件 (test event) 📌

這一整包是 Lambda 在「後台」收到的 event 格式：

```
json複製程式碼  
  
{  
  "resource": "/audio-callback",  
  "path": "/audio-callback",  
  "httpMethod": "POST",  
  "headers": { ... },  
  ...  
  "body": "{\"code\":200,\"data\":{\"...},\"msg\":\"All generated successfully.\"}"  
  "isBase64Encoded": false  
}
```

2. Flask route :

```
python複製程式碼  
  
@app.route('/audio-callback', methods=['POST'])  
def audio_callback():  
    ...  
    data = request.json  
    suno_items = data["data"]["data"]  
    ...
```

重點一：Flask 看不到整個 event，只看得到「body 裡那一段」

你現在這個架構是：

API Gateway (REST · Lambda proxy) → Lambda → awsgi → Flask

在這串流程裡：

- Lambda 收到的 `event` 才是你貼的那一大包 (有 `resource`、`httpMethod`、`body` 那個)
- `awsgi` 會幫你把 `event["body"]` 變成 Flask 的 `request.data` / `request.json`

- 所以在 Flask 的 `audio_callback()` 裡：

```
python
```

[複製程式碼](#)

```
data = request.json
```

這個 `data` 應該只會長這樣（也就是 `event["body"]` 解 JSON 後）：

```
json
```

[複製程式碼](#)

```
{
  "code": 200,
  "data": {
    "callbackType": "complete",
    "data": [ ... 很多音樂資料 ... ],
    "task_id": "2b00f76847c403fe9051b70c95bc27da"
  },
  "msg": "All generated successfully."
}
```

這時你的這行是合法的：

```
python
```

[複製程式碼](#)

```
suno_items = data["data"]["data"]
#           ↑外層   ↑內層
```

重點二：為什麼會噴 `KeyError('data')` ？

這種錯誤只會在：

```
python
```

[複製程式碼](#)

```
data = request.json          # 得到的是一個 dict
suno_items = data["data"]    # 但這個 dict 裡根本沒有 key "data"
```

才會出現 `KeyError('data')`。

那什麼情況 `data` 裡不會有 `"data"` key ？

👉 你把整包 **Lambda test event** 貼進 Postman 當 body 送了。

也就是你可能在 Postman 裡送的是這樣（錯誤示範 ❌）：

json

 複製程式碼

```
{
  "resource": "/audio-callback",
  "path": "/audio-callback",
  "httpMethod": "POST",
  ...
  "body": "{\"code\":200,\"data\":{\"...}}",
  ...
}
```

這樣 Flask `request.json` 拿到的就是上面這一整包，最外層 key 有：

- `resource`
- `path`
- `httpMethod`
- `headers`
- `body`
- ...

👉 沒有 `"data"` 這個 key，所以 `data["data"]` 直接爆 `KeyError('data')`，然後被你這段 catch：

python

 複製程式碼

```
except Exception as e:
    print("callback error:", e)
    return jsonify({"error": str(e)}), 500
```

就變成你看到的：

json

 複製程式碼

```
{"error": "'data'"}
```

重點三：Postman 正確要怎麼送？

在 Postman 裡，你要送的是「Suno 真正 callback 給你的 body」那一段，也就是 Lambda event 裡的 `body` 解析後的內容，而不是整個 test event。

✅ 正確做法 (Postman 設定)

1. Method：POST

2. URL：你的 API Gateway https，例如

`https://xxxxxx.execute-api.ap-northeast-1.amazonaws.com/prod/audio-callback`

3. Headers：

- `Content-Type: application/json`

4. Body → raw → JSON，貼 這段（注意，這才是內容本體）：

json

複製程式碼

```
{
  "code": 200,
  "data": {
    "callbackType": "complete",
    "data": [
      {
        "audio_url": "https://musicfile.api.box/MjU4ZDA4YjUtNDJmNi00ZDFhLTg1MGMtN",
        "createTime": 1763910096539,
        "duration": 107.92,
        "id": "258d08b5-42f6-4d1a-850c-46ecbb786d03",
        "image_url": "https://musicfile.api.box/MjU4ZDA4YjUtNDJmNi00ZDFhLTg1MGMtN",
        "model_name": "chirp-v3-5",
        "prompt": "",
        "source_audio_url": "https://cdn1.suno.ai/258d08b5-42f6-4d1a-850c-46ecbb7",
        "source_image_url": "https://cdn2.suno.ai/image_258d08b5-42f6-4d1a-850c-4",
        "source_stream_audio_url": "https://cdn1.suno.ai/258d08b5-42f6-4d1a-850c-",
        "stream_audio_url": "https://musicfile.api.box/MjU4ZDA4YjUtNDJmNi00ZDFhLT",
        "tags": "古典",
        "title": "宁静钢琴冥想"
      },
      {
        "audio_url": "https://musicfile.api.box/YWVjYzIxYzAtNDk1MC00MTVmLTg1ODUtO",
        "createTime": 1763910096539,
        "duration": 168.36,
        "id": "aecc91c0-4950-415f-8585-81f74c491ddc",
        "image_url": "https://musicfile.api.box/YWVjYzIxYzAtNDk1MC00MTVmLTg1ODUtO",
        "model_name": "chirp-v3-5",
        "prompt": "",
        "source_audio_url": "https://cdn1.suno.ai/aecc91c0-4950-415f-8585-81f74c4",
        "source_image_url": "https://cdn2.suno.ai/image_aecc91c0-4950-415f-8585-8",
        "source_stream_audio_url": "https://cdn1.suno.ai/aecc91c0-4950-415f-8585-",
        "stream_audio_url": "https://musicfile.api.box/YWVjYzIxYzAtNDk1MC00MTVmLT",
        "tags": "古典",
        "title": "宁静钢琴冥想"
      }
    ]
  }
}
```

```
],  
  "task_id": "2b00f76847c403fe9051b70c95bc27da"  
},  
  "msg": "All generated successfully."  
}
```

這樣進到 Flask 之後：

python

 複製程式碼

```
data = request.json  
# data["data"]["data"] 就會是上面那個陣列，取得到  
suno_items = data["data"]["data"]
```

就不會再 KeyError 了。

建議你加一行 debug，確認自己看到的是什麼

在你的 route 一開始加這個：

python

 複製程式碼

```
@app.route('/audio-callback', methods=['POST'])  
def audio_callback():  
    print("★ 進到 audio_callback")  
    print("raw body:", request.data)  
    print("json body:", request.get_json(silent=True))  
    ...
```

然後去 CloudWatch Logs 看實際印出來的 `json body` 長什麼樣。

如果印出來跟你以為的不一樣，就知道是哪個環節送錯了。

小小額外提醒

你現在的 `for item in suno_items:` 裡面，最後 `return` 寫在迴圈裡：

python

 複製程式碼

```
for item in suno_items:  
    ...  
    return jsonify({
```

```

    "success": True,
    "download_url": presigned_url,
    "reply": "Flask 收到音樂prompt"
})

```

這樣只會處理 **第一首歌就 return**，後面的 item 都不會跑到。

之後如果你要一次處理多首，可以改成把結果存到 list，迴圈完再一起回傳。

總結一句話：

Postman 測的時候，body 只要送「真正的 callback JSON」，不要把整個 Lambda test event 那包貼進去；你的程式才找得到 `data["data"]["data"]`，不會再噴

`''data''`。



```

from flask import Flask, request, jsonify,
send_from_directory
from dotenv import load_dotenv
import os, io, uuid, datetime
import requests
from PIL import Image
import awsgi
import boto3
from db import insert_record, get_history
import json

load_dotenv()
HuggingFace_Token =
os.getenv("HUGGINGFACE_TOKEN")
Suno_Token = os.getenv("SUNO_TOKEN")
S3_BUCKET_NAME = os.getenv("S3_BUCKET_NAME")
HuggingFace_URL = "https://router.huggingface.co/hf-
inference/models/stabilityai/stable-diffusion-xl-base-
1.0"
SUNO_URL =
"https://api.sunoapi.org/api/v1/generate"

app = Flask(__name__)

```

```
s3 = boto3.client("s3") # 建立 s3 client

@app.route("/generate-image", methods=["POST"])
def generate_image():
    try:
        data = request.json
        prompt = data.get("prompt", "沒有收到圖片 prompt")
        user_id = data.get("user_id", "unknown")

        if not prompt:
            return jsonify({"success": False, "error": "缺少 prompt"}), 400

        headers = {"Authorization": f"Bearer {HuggingFace_Token}"}
        payload = {"inputs": prompt}

        response = requests.post(HuggingFace_URL,
                                  headers=headers, json=payload, timeout=60)

        if response.status_code != 200:
            return jsonify({
                "success": False,
                "error": f"Hugging Face API 錯誤: {response.status_code}",
                "details": response.text
            }, 500)

        content_type = response.headers.get('Content-Type', '')

        if 'application/json' in content_type:
            error_data = response.json()
            return jsonify({
                "success": False,
                "error": "Hugging Face 回傳錯誤",
                "details": error_data
            }, 500)

        if 'image' not in content_type:
            return jsonify({
```

```
"success": False,
"error": f"非預期的回傳格式: {content_type}"
}), 500

# 嘗試解析圖片
try:
    image =
Image.open(io.BytesIO(response.content))
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"圖片解析失敗: {str(e)}"
    }), 500

# 存到 S3
timestamp =
datetime.datetime.now().strftime("%Y%m%d%H%M%S")
filename = f"
{timestamp}_{uuid.uuid4().hex[:8]}.png"
s3_key = f"Images/{filename}"

try:
    s3.put_object(
        Bucket=S3_BUCKET_NAME,
        Key=s3_key,
        Body=response.content,    # 直接用原始 bytes
        ContentType="image/png"
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"上傳 S3 失敗: {str(e)}"
    }), 500

public_url=f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"
local_url=f"s3://{S3_BUCKET_NAME}/{s3_key}"
# 存進 DynamoDB
try:
    result = insert_record(
        user_id=user_id,
```



```
        prompt=prompt,
        file_url=public_url,
        record_type="image",
        status="success"
    )
except Exception as e:
    return jsonify({
        "statusCode": 500,
        "body": json.dumps({
            "message": "✖ 插入失敗",
            "error": str(e)
        })
    })

# 提供可存取的 url
try:
    presigned_url = s3.generate_presigned_url(
        ClientMethod="get_object",
        Params={
            "Bucket": S3_BUCKET_NAME,
            "Key": s3_key
        },
        ExpiresIn=3600 # 有效時間秒數
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"產生下載網址失敗: {str(e)}"
    }), 500

return jsonify({
    "success": True,
    "download_url": presigned_url, # 👉 這是 S3
    "reply": f"Flask 收到圖片 prompt: {prompt}"
})

except Exception as e:
    return jsonify({
        "success": False,
        "error": str(e)
    }), 500
```

```
@app.route('/generate-audio', methods=['POST'])
def generate_audio():
    try:
        print("=== /generate-audio 被呼叫 ===")

        data = request.json
        print("收到的 JSON:", data)

        prompt = data.get('prompt')
        # discord_channel_id =
        data.get('discord_channel_id')
        # discord_user_id = data.get('discord_user_id')

        if not prompt:
            print("❌ 缺少 prompt")
            return jsonify({"success": False, "error": "缺少 prompt"}), 400

        payload = {
            "prompt": prompt,
            "style": "古典",
            "title": "AI Generated Music",
            "customMode": True,
            "instrumental": True,
            "model": "V3_5",
            "callbackUrl": "https://xpapysqd2i.execute-
api.us-east-1.amazonaws.com/prod/audio-callback"
        }

        print("Suno payload:", payload)

        headers = {
            "Authorization": f"Bearer {Suno_Token}",
            "Content-Type": "application/json"
        }

        print("正在呼叫 Suno API...")
        response = requests.post(SUNO_URL,
            json=payload, headers=headers, timeout=60)

        print("Suno 回應狀態:", response.status_code)
        print("Suno 回應內容:", response.text)
```

```
if response.status_code != 200:
    return jsonify({
        "success": False,
        "error": f"Suno API 錯誤:
{response.status_code}",
        "details": response.text
    }), 500

suno_resp = response.json()
print("解析後 Suno JSON:", suno_resp)

# ★ 抓 taskId ( Suno 回傳格式有時不同 )
suno_request_id = (
    suno_resp.get("id")
    or suno_resp.get("requestId")
    or suno_resp.get("data", {}).get("taskId")
)

if not suno_request_id:
    print("✗ Suno 沒有回傳 taskId")
    return jsonify({
        "success": False,
        "error": "Suno 未回傳 taskId"
    }), 500

# 回傳給 Discord bot · 可以用來顯示“開始生成中”
return jsonify({
    "success": True,
    "task_id": suno_request_id,
    "message": "音樂生成中"
})

except Exception as e:
    print("✗ Flask 在 /generate-audio 發生錯誤:", e)
    return jsonify({"success": False, "error": str(e)}), 500

@app.route('/audio-callback', methods=['POST'])
def audio_callback():
    print("★ 進到 audio_callback")
    try:
        print("=== 收到 Suno callback ===")
        data = request.json
```

```
suno_items = data["data"]["data"]

for item in suno_items:
    audio_url = item.get("source_audio_url")
    music_id = item["id"]

    if not audio_url:
        print("❌ 沒有 audio_url")
        return jsonify({
            "success": False,
            "error": "Suno 沒有回傳 audio_url"
        }), 500

    # 下載音檔
    mp3 = requests.get(audio_url).content

    # 上傳 S3
    timestamp =
datetime.datetime.now().strftime("%Y%m%d%H%M%S")
    filename = f"
{timestamp}_{uuid.uuid4().hex[:8]}.mp3"
    s3_key = f"Audios/{filename}"

    try:
        s3.put_object(
            Bucket=S3_BUCKET_NAME,
            Key=s3_key,
            Body=mp3,    # 直接用原始 bytes
            ContentType="audio/mpeg"
        )
    except Exception as e:
        return jsonify({
            "success": False,
            "error": f"上傳 S3 失敗: {str(e)}"
        }), 500

    public_url=f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"
    local_url=f"s3://{S3_BUCKET_NAME}/{s3_key}"

    # 存進 DynamoDB
```

```
try:
    result = insert_record(
        user_id="no user id",
        prompt="no prompt",
        file_url=public_url,
        record_type="audio",
        status="success"
    )
except Exception as e:
    return jsonify({
        "statusCode": 500,
        "body": json.dumps({
            "message": "✖ 插入失敗",
            "error": str(e)
        })
    })
# 提供可存取的 url
try:
    presigned_url = s3.generate_presigned_url(
        ClientMethod="get_object",
        Params={
            "Bucket": S3_BUCKET_NAME,
            "Key": s3_key
        },
        ExpiresIn=3600 # 有效時間秒數
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"產生下載網址失敗: {str(e)}"
    }), 500

return jsonify({
    "success": True,
    "download_url": presigned_url, # 👉 這是 S3
    "reply": "Flask 收到音樂prompt"
})

except Exception as e:
    print("callback error:", e)
    return jsonify({"error": str(e)}), 500
```

```
@app.route("/history", methods=["GET"])
def history():
    try:
        user_id = request.args.get("user_id") # 從 URL
        query 取得 user_id

        records = get_history(user_id)

        return jsonify({
            "success": True,
            "records": records # 形式 : [{ "prompt": "...",
            "file_url": "... }], ...]
        })

    except Exception as e:
        return jsonify({"success": False, "error": str(e)}), 500

if __name__ == '__main__':
    print("🚀 Flask 伺服器啟動中...")
    app.run(host='0.0.0.0', port=5000, debug=True)

import discord
from discord.ext import commands
import logging
from dotenv import load_dotenv
import os
import io
import aiohttp
import requests

load_dotenv()
DC_token
="MTQzNTI3MDk4NDc4NDU0Nzg0Mg.Glnp2b.JOi1Ke
8hUGHyH_W0tp2q-D0WCtFiDglyUCZi3U"
# DC_token = os.getenv('DISCORD_TOKEN')
# FLASK_API_URL = os.getenv('FLASK_API_URL',
'http://localhost:5000')
FLASK_API_URL = "https://xpapysqd2i.execute-api.us-
east-1.amazonaws.com/prod"

handler = logging.FileHandler(filename='discord.log',
encoding='utf-8', mode='w')
intent = discord.Intents.default()
```

```
intent.message_content = True
intent.members = True

bot = commands.Bot(command_prefix='/',
intents=intent)

@bot.event
async def on_ready():
    print(f"We are ready to go in, {bot.user.name}")

@bot.command()
async def hello(ctx):
    await ctx.send(f"Hello {ctx.author.mention}!")

@bot.command()
async def create_photo(ctx, *, prompt: str = "未輸入
prompt"):
    try:
        payload = {"prompt": prompt, "user_id":
str(ctx.author.id)}
        # response =
requests.post("http://localhost:5000/generate-image",
json=payload, timeout=60)
        response = requests.post(f"
{FLASK_API_URL}/generate-image", json=payload,
timeout=60)
        data = response.json()
        """
        if not data.get("success"):
            err = data.get("error") or "未知錯誤"
            details = data.get("details")
            msg = f"❌ Flask 回傳錯誤: {err}"
            if details:
                msg += f"\n 📄 詳細: {details}"
            await ctx.send(msg)
            return
        """
        if not data.get("success"):
            await ctx.send(f"❌ Flask 回傳錯誤:
{data.get('error', '未知錯誤')}")
            return
```

```
file_url = data.get("download_url")
if not file_url:
    await ctx.send("❌ Flask 沒有回傳檔案 URL")
    return

file_resp = requests.get(file_url)
if file_resp.status_code == 200:
    with io.BytesIO(file_resp.content) as f:
        await ctx.send(file=discord.File(f,
filename="generated.png"))
    else:
        await ctx.send("❌ 下載圖片失敗")

except Exception as e:
    await ctx.send(f"❌ 呼叫 Flask 失敗: {e}")

# /create_audio 指令
@bot.command()
async def create_audio(ctx, *, prompt: str = "未輸入 prompt"):
    # await ctx.send("這是生成音樂的功能")
    try:
        payload = {"prompt": prompt}
        response = requests.post(f"
{FLASK_API_URL}/generate-audio", json=payload,
timeout=10)
        data = response.json()

        if not data.get("success"):
            await ctx.send(f"❌ Flask 回傳錯誤:
{data.get('reply', '未知錯誤')}")
            return

    ## 2 取得 local_url
    # file_url = data.get("local_url")
    # if not file_url:
    #     await ctx.send("❌ Flask 沒有回傳檔案 URL")
    #     return

    # 3 從 URL 下載檔案內容
    file_resp = data.get("download_url")
    # file_content = file_resp.text # 文字檔使用 text
```



```
# 📌 將內容回 Discord
await ctx.send(f"✅ 從 Flask 取得檔案內容:\n{file_resp}")

except Exception as e:
    await ctx.send(f"❌ 呼叫 Flask 失敗: {e}")

@bot.command()
async def history(ctx):
    try:
        # 自動帶入 Discord 使用者 ID
        params = {"user_id":str(ctx.author.id)}
        response = requests.get(f"{FLASK_API_URL}/history", params = params,
                                timeout=10)
        data = response.json()

        if not data.get("success"):
            await ctx.send(f"❌ Flask 回傳錯誤: {data.get('error', '未知錯誤')}")
            return

        records = data.get("records", [])
        if not records:
            await ctx.send(f"👤 使用者 {ctx.author.id} 沒有任何紀錄。")
            return

        lines = []
        for idx, r in enumerate(records, start=1):
            prompt = r.get("prompt", "(無prompt)")
            url = r.get("file_url", "(無網址)")
            dataType = r.get("type", "(無形別)")
            lines.append(f"{idx}. {dataType} : {prompt} → {url}")

        msg = "\n".join(lines)

        # 分段輸出避免超過 Discord 限制
        if len(msg) > 1900:
            chunks = [msg[i:i+1900] for i in range(0, len(msg), 1900)]
```

```

for chunk in chunks:
    await ctx.send(f" 📄 你的紀錄:\n{chunk}",
suppress_embeds=True)
else:
    await ctx.send(f" 📄 你的紀錄:\n{msg}",
suppress_embeds=True)

except Exception as e:
    await ctx.send(f" ❌ 查詢 DB 失敗: {e}")

bot.run(DC_token, log_handler=handler,
log_level=logging.DEBUG)

```

剛剛在discord 打/create_audio 爆炸，直接回傳 ❌ Flask
回傳錯誤: 未知錯誤
· 但用testevent卻沒錯，甚麼問題

已思考 1m 34s >

先講結論：

你在 Lambda Console 用 **test event** 測的是 `/audio-callback`，那一條路是「Suno 生完音樂之後招呼你」；

但 Discord `/create_audio` 打的是 `/generate-audio`，兩條完全不同的流程，所以「callback 測 OK」不代表「generate-audio 也 OK」。

現在 `/create_audio` 會回：

❌ Flask 回傳錯誤: 未知錯誤

是因為：

python

複製程式碼

```

if not data.get("success"):
    await ctx.send(f" ❌ Flask 回傳錯誤: {data.get('reply', '未知錯誤')}")

```

但你的 Flask `/generate-audio` 在出錯時回的是：

python

複製程式碼

```

return jsonify({
    "success": False,

```

```
"error": "Suno 未回傳 taskId" # 或 "Suno API 錯誤: xxx"
}), 500
```

注意：沒有 `reply` 這個欄位，只有 `error`，所以 Discord 這行：

python

複製程式碼

```
data.get('reply', '未知錯誤')
```

拿不到 `reply`，就用預設 `"未知錯誤"`。

所以真實錯誤訊息其實藏在 `data["error"]`，只是你沒有把它印出來而已。

一、為什麼 test event 會成功？

你給的 test event 是這個（重點節錄）：

json

複製程式碼

```
{
  "resource": "/audio-callback",
  "path": "/audio-callback",
  "httpMethod": "POST",
  ...
  "body": "{\"code\":200,\"data\":{\"...},\"msg\":\"All generated successfully.\"}"
}
```

這個是「模擬 Suno 打回 `/audio-callback`」的事件，進的是這個函式：

python

複製程式碼

```
@app.route('/audio-callback', methods=['POST'])
def audio_callback():
    ...
```

它跟 Discord `/create_audio` 走的 `/generate-audio` 完全不同：

python

複製程式碼

```
@app.route('/generate-audio', methods=['POST'])
def generate_audio():
    ...
```

所以：

- ☒ 你已經證明：**callback 路徑** `/audio-callback` 處理 OK
- 但現在壞的是：前面那一步 `/generate-audio` 呼叫 Suno 時出錯

二、`/generate-audio` 中可能出錯的點

看你的 `/generate-audio`：

python

 複製程式碼

```
@app.route('/generate-audio', methods=['POST'])
def generate_audio():
    try:
        ...
        data = request.json
        prompt = data.get('prompt')

        if not prompt:
            return jsonify({"success": False, "error": "缺少 prompt"}), 400

        payload = {
            "prompt": prompt,
            "style": "古典",
            "title": "AI Generated Music",
            "customMode": True,
            "instrumental": True,
            "model": "V3_5",
            "callbackUrl": "https://xpapysqd2i.execute-api.us-east-1.amazonaws.co
        }

        headers = {
            "Authorization": f"Bearer {Suno_Token}",
            "Content-Type": "application/json"
        }

        response = requests.post(SUNO_URL, json=payload, headers=headers, timeout

        if response.status_code != 200:
            return jsonify({
                "success": False,
                "error": f"Suno API 錯誤: {response.status_code}",
                "details": response.text
```

```

    }), 500

    suno_resp = response.json()

    suno_request_id = (
        suno_resp.get("id")
        or suno_resp.get("requestId")
        or suno_resp.get("data", {}).get("taskId")
    )

    if not suno_request_id:
        return jsonify({
            "success": False,
            "error": "Suno 未回傳 taskId"
        }), 500

    return jsonify({
        "success": True,
        "task_id": suno_request_id,
        "message": "音樂生成中"
    })

except Exception as e:
    return jsonify({"success": False, "error": str(e)}), 500

```

會回 `success = False` 的幾種情況：

1. `prompt` 是空字串 / None → "缺少 prompt"
2. Suno 回應不是 200 → "Suno API 錯誤: 401/400/500..." + details
3. Suno 回的是 200，但 JSON 裡沒有你要的 `id` / `requestId` / `data.taskId` → "Suno 未回傳 taskId"
4. 其他 Exception (例如 requests timeout、解析 JSON 失敗) → `error = str(e)`

你現在在 Discord 只看 `reply`，所以這些訊息都沒顯示出來。

三、先把真正的錯誤抓出來看

1 修改 Discord `/create_audio` (建議先改這樣)

python

 複製程式碼

```

@bot.command()
async def create_audio(ctx, *, prompt: str="未輸入prompt"):
    try:
        payload = {"prompt": prompt}
        response = requests.post(f"{FLASK_API_URL}/generate-audio", json=payload,
                                data = response.json())

        if not data.get("success"):
            await ctx.send(f"❌ Flask 回傳錯誤: {data.get('error') or data}")
            return

        # 這裡暫時只看 task_id 就好
        task_id = data.get("task_id")
        await ctx.send(f"✅ Suno 開始生成中, task_id: {task_id}")

    except Exception as e:
        await ctx.send(f"❌ 呼叫 Flask 失敗: {e}")

```

重點：把 `data.get('reply', '未知錯誤')` 改成 `data.get('error') or data`，先看到真實錯誤再說。

2 在 Flask `/generate-audio` 裡多印幾行 log

python

 複製程式碼

```

@app.route('/generate-audio', methods=['POST'])
def generate_audio():
    try:
        print("=== /generate-audio 被呼叫 ===")
        print("request.json:", request.get_json(silent=True))

        ...

        print("Suno 回應狀態:", response.status_code)
        print("Suno 回應內容:", response.text)

        ...

    except Exception as e:
        print("❌ Flask 在 /generate-audio 發生錯誤:", e)
        return jsonify({"success": False, "error": str(e)}), 500

```

然後去 **CloudWatch Logs** 看真實訊息是什麼：

- 如果是 `Suno API 錯誤: 401` → 很可能 `Suno_Token` 錯 / 沒設好環境變數
- 如果是 `Suno API 錯誤: 400` → payload 格式不對 (參數名或 model 名)
- 如果是 `Suno 未回傳 taskId` → Suno 回的 JSON 結構跟你預期不一樣 (要對照 Suno 文件)

四、用 Postman 直接打一遍 `/generate-audio` 也很好 debug

在 Postman :

- Method: `POST`
- URL: `https://xpapysqd2i.execute-api.us-east-1.amazonaws.com/prod/generate-audio`
- Headers:
 - `Content-Type: application/json`
- Body (raw, JSON) :

json

複製程式碼

```
{
  "prompt": "幫我生一首輕柔鋼琴曲"
}
```

看回來的 JSON :

- 如果是 :

json

複製程式碼

```
{
  "success": true,
  "task_id": "xxxxx",
  "message": "音樂生成中"
}
```

→ 那就沒問題，Discord 端要處理的就是 `task_id` (音樂實際檔案等 Suno callback)

- 如果是 :

json

複製程式碼

```
{
  "success": false,
  "error": "Suno API 錯誤: 401",
}
```

```
"details": "..."  
}
```

→ 問題出在 Suno 那邊 (token / 參數) , 跟 Discord 無關。

五、順便說：/create_audio 的設計跟現在的後端不太 match

現在你的流程是「非同步」：

1. /generate-audio → 呼叫 Suno , 拿一個 taskId → 立即回傳「音樂生成中」
2. Suno 生完之後 , 打 HTTP POST 到 /audio-callback
3. /audio-callback 下載音檔、上 S3、存 DB

所以 /generate-audio 不會也不應該回 download_url ;

它只負責「建立任務」, 真正的音檔 URL 應該從：

- callback 時推回 Discord (例如你從 audio-callback 去 call Discord Webhook) , 或
- 之後讓使用者 /history 查自己的紀錄 , 再去點 URL 。

你現在 Discord 裡 /create_audio 的後半段是：

python

複製程式碼

```
file_resp = data.get("download_url")  
await ctx.send(f"✅ 從 Flask 取得檔案內容:\n{file_resp}")
```

這本來就拿不到東西 , 因為 /generate-audio 根本沒有 download_url 這個欄位。

所以比較合理的 flow 是：

python

複製程式碼

```
if not data.get("success"):  
    ... # 顯示 error  
else:  
    task_id = data.get("task_id")  
    await ctx.send(f"✅ 已送出 Suno 任務, task_id: {task_id}\n完成後會有另一個訊息通
```

然後等 Suno callback 之後 , 再設計你要怎麼通知使用者。

六、超重要：你的 Discord Bot Token 已經曝光了

你現在貼出來的這行：

```
python
```

[複製程式碼](#)

```
DC_token = "MTQzNTI3MDk4NDc4NDU0Nzg0Mg.GInp2b.J0i1Ke8hUGHyH_W0tp2q-D0WCtFiDglyUCZi
```

這是 **完整的 Discord Bot Token**，等於別人拿到你的 bot 控制權。

務必馬上做這兩件事：

1. 去 Discord Developer Portal → 到你這個 Bot → **Regenerate Token**
2. 把新的 Token 放到 `.env`，改成用 `os.getenv('DISCORD_TOKEN')`，不要再寫死在程式裡

```
python
```

[複製程式碼](#)

```
load_dotenv()
DC_token = os.getenv('DISCORD_TOKEN')
...
bot.run(DC_token, log_handler=handler, log_level=logging.DEBUG)
```

不然真的有人拿去亂搞你的伺服器，會很麻煩。

總結

- test event 只測了 `/audio-callback`，沒測到 `/generate-audio`，所以「沒錯」不代表前半段也 OK。
- `/create_audio` 顯示「未知錯誤」，是因為你沒印出 `data["error"]`，真實錯誤被藏起來。
- 先改 Discord 那段，把 `error` 顯示出來，再搭配 CloudWatch log，看 Suno 回了什麼，就能定位是 token、參數還是 JSON 結構的問題。
- `/generate-audio` 設計是非同步任務，不會回 `download_url`，Discord 那邊邏輯要改成處理 `task_id`。

你可以先改完 `/create_audio` 跟 `/generate-audio` 的 log，跑一次，

把 **Flask / Lambda log** 中印出來的 `Suno 回應狀態` + `Suno 回應內容` 貼給我，我可以幫你看 Suno 那邊到底在抱怨什麼。

[複製](#) [分享](#) [評論](#) [收藏](#) [刷新](#) [更多](#)

```

{
  "resource": "/audio-callback",
  "path": "/audio-callback",
  "httpMethod": "POST",
  "headers": {
    "Content-Type": "application/json"
  },
  "multiValueHeaders": {
    "Content-Type": [
      "application/json"
    ]
  },
  "queryStringParameters": null,
  "multiValueQueryStringParameters": null,
  "pathParameters": null,
  "stageVariables": null,
  "requestContext": {
    "path": "/audio-callback",
    "stage": "prod",
    "identity": {
      "sourceIp": "127.0.0.1",
      "userAgent": "lambda-test-event"
    }
  },
  "body": "{\"code\":200,\"data\":{
  \"callbackType\": \"complete\", \"data\":
  [{\"audio_url\": \"https://musicfile.api.box/MjU4ZDA4Yj
  UtNDJmNi00ZDFhLTg1MGMtNDZIY2JiNzg2ZDAz.mp3\\
  \", \"createTime\": 1763910096539, \"duration\": 107.92, \"i
  d\": \"258d08b5-42f6-4d1a-850c-
  46ecbb786d03\", \"image_url\": \"https://musicfile.api.b
  ox/MjU4ZDA4YjUtNDJmNi00ZDFhLTg1MGMtNDZIY2Ji
  Nzg2ZDAz.jpeg\", \"model_name\": \"chirp-v3-
  5\", \"prompt\": \"\", \"source_audio_url\": \"https://cdn1.
  suno.ai/258d08b5-42f6-4d1a-850c-
  46ecbb786d03.mp3\", \"source_image_url\": \"https://cd
  n2.suno.ai/image_258d08b5-42f6-4d1a-850c-
  46ecbb786d03.jpeg\", \"source_stream_audio_url\": \"htt
  ps://cdn1.suno.ai/258d08b5-42f6-4d1a-850c-
  46ecbb786d03.mp3\", \"stream_audio_url\": \"https://m
  usicfile.api.box/MjU4ZDA4YjUtNDJmNi00ZDFhLTg1MG

```

```
MtNDZlY2JiNzg2ZDAz\", \"tags\": \"古典\", \"title\": \"宁静
钢琴冥想\"},
{ \"audio_url\": \"https://musicfile.api.box/YWVjYzIxYzAt
NDk1MC00MTVmLTg1ODUtODFmNzRjNDkxZGRj.mp3
\", \"createTime\": 1763910096539, \"duration\": 168.36, \"
id\": \"aecc91c0-4950-415f-8585-
81f74c491ddc\", \"image_url\": \"https://musicfile.api.bo
x/YWVjYzIxYzAtNDk1MC00MTVmLTg1ODUtODFmNzR
jNDkxZGRj.jpeg\", \"model_name\": \"chirp-v3-
5\", \"prompt\": \"\", \"source_audio_url\": \"https://cdn1.
suno.ai/aecc91c0-4950-415f-8585-
81f74c491ddc.mp3\", \"source_image_url\": \"https://cd
n2.suno.ai/image_aecc91c0-4950-415f-8585-
81f74c491ddc.jpeg\", \"source_stream_audio_url\": \"htt
ps://cdn1.suno.ai/aecc91c0-4950-415f-8585-
81f74c491ddc.mp3\", \"stream_audio_url\": \"https://mu
sicfile.api.box/YWVjYzIxYzAtNDk1MC00MTVmLTg1OD
UtODFmNzRjNDkxZGRj\", \"tags\": \"古典\", \"title\": \"宁
静钢琴冥想
\"}], \"task_id\": \"2b00f76847c403fe9051b70c95bc27da
\", \"msg\": \"All generated successfully.\"},
  \"isBase64Encoded\": false
}
```

找user_id跟prompt

已思考 30s >

先講關鍵：

這包 Suno callback 裡根本沒有 `user_id`，只有（空的）`prompt` 跟 `task_id`。

所以「找 `user_id`」這題，答案是：現在這個 JSON 找不到，要靠你自己在前一段
`/generate-audio` 存起來，再用 `task_id` 去對。

1. 先看這包 JSON 的結構

`event[\"body\"]` 解析後長這樣（縮排一下）：

json

 複製程式碼

```
{
  \"code\": 200,
  \"data\": {
```

```

"callbackType": "complete",
"data": [
  {
    "audio_url": "https://musicfile.api.box/....mp3",
    "createTime": 1763910096539,
    "duration": 107.92,
    "id": "258d08b5-42f6-4d1a-850c-46ecbb786d03",
    "image_url": "https://musicfile.api.box/....jpeg",
    "model_name": "chirp-v3-5",
    "prompt": "",
    "source_audio_url": "https://cdn1.suno.ai/258d08b5-42f6-4d1a-850c-46ecbb7
    ...
    "tags": "古典",
    "title": "宁静钢琴冥想"
  },
  {
    "audio_url": "...第二首...",
    "prompt": "",
    ...
  }
],
"task_id": "2b00f76847c403fe9051b70c95bc27da"
},
"msg": "All generated successfully."
}

```

所以在你的 Flask callback 裡：

```
python
```

 複製程式碼

```

data = request.json          # ← 就是上面這個 dict
suno_items = data["data"]    # ← 兩首歌的 list
task_id = data["data"]["task_id"]  # ← 這次任務的 id

```

如果你要拿「Suno 那邊的 prompt」：

```
python
```

 複製程式碼

```

for item in suno_items:
    prompt = item.get("prompt")  # 但你現在看到的是空字串 ""

```

結論：

- ☒ 可以拿到：`task_id`、每首歌自己的 `id`、`title`、`tags`、`audio_url` ...
- ☒ 完全沒有 `user_id` (Discord 那個作者 ID 你沒塞進去)
- `prompt` 欄位目前是空字串 (Suno callback 給你的就是 "")

2. 那我要怎麼在 callback 拿到 `user_id` + 原本的 `prompt` ?

這種情況標準做法是：

前面 `/generate-audio` 呼叫 Suno 拿到 `taskId` 的時候，
自己在 DB 建一筆「`task` ↔ `user_id, prompt`」對應關係，
callback 回來用 `task_id` 去查。

第一步：在 `/generate-audio` 存 mapping

你目前 `/generate-audio` 長這樣 (重點)：

python

 複製程式碼

```
@app.route('/generate-audio', methods=['POST'])
def generate_audio():
    ...
    data = request.json
    prompt = data.get('prompt')
    # (建議順便傳 user_id 進來)
    user_id = data.get('user_id', 'unknown')

    ...

    suno_resp = response.json()
    suno_request_id = (
        suno_resp.get("id")
        or suno_resp.get("requestId")
        or suno_resp.get("data", {}).get("taskId")
    )

    if not suno_request_id:
        return jsonify({
            "success": False,
            "error": "Suno 未回傳 taskId"
        }), 500
```

你可以在這裡多存一筆 DB (可以用你自己的 `insert_record` , 或開一張專門的 table) , 例如假設你有一個 `save_task_mapping(task_id, user_id, prompt)` :

python

 複製程式碼

```
# 取得 taskId 後, 先存到 DB
try:
    save_task_mapping(
        task_id=suno_request_id,
        user_id=user_id,
        prompt=prompt
    )
except Exception as e:
    print("✖ 存 task mapping 失敗:", e)
    # 看你要不要直接當錯誤處理
```

這樣 DB 會有一筆類似 :

task_id	user_id	prompt
2b00f76847c403fe9051b70c95bc27da	123456789	「輕柔鋼琴冥想音樂」

第二步：在 `/audio-callback` 用 `task_id` 把資料撈回來

在 callback 裡你先抓 `task_id` :

python

 複製程式碼

```
@app.route('/audio-callback', methods=['POST'])
def audio_callback():
    print("★ 進到 audio_callback")
    try:
        data = request.json
        suno_items = data["data"]["data"]
        task_id = data["data"].get("task_id") # ★ 這裡拿到

        # 從 DB 查回 user_id & prompt
        mapping = get_task_mapping(task_id) # 自己實作, 例如查 DynamoDB
        user_id = mapping["user_id"]
        prompt = mapping["prompt"]

        for item in suno_items:
```

```
audio_url = item.get("source_audio_url")
music_id = item["id"]
...
# 存進 DynamoDB 時就可以用真的 user_id 和 prompt
result = insert_record(
    user_id=user_id,
    prompt=prompt,
    file_url=public_url,
    record_type="audio",
    status="success"
)
...
```

這樣你 callback 就可以知道：

- 這次 Suno 的任務原本是哪個 Discord 使用者發的 (`user_id`)
- 他當初打的 prompt 是什麼 (`prompt`)

3. 如果你真的想「從 callback JSON 直接拿」，目前做不到

因為現在 callback 的 body 裡：

- `prompt` 是空字串 ""
- `user_id` 完全不存在

你可以做一些小 hack，例如：

1. 在呼叫 Suno 的時候就把 `user_id` / `prompt` 塞進 `title` 或 `tags`：

python

 複製程式碼

```
payload = {
    "prompt": prompt,
    "style": "古典",
    "title": f"[{ctx.author.id}] {prompt[:20]}",
    "tags": f"user={ctx.author.id}",
    ...
}
```

2. 那 callback 裡就可以從 `item["title"]` / `item["tags"]` 把它 parse 出來。

但這種屬於比較 hack 的方式，比較乾淨、可維護的是用 DB 以 `task_id` 當 key 做 mapping。

4. 小結

- 你給的這包 JSON 裡：
 - `task_id`：在 `data["data"]["task_id"]`
 - 每首歌的 `prompt`：在 `data["data"]["data"][i]["prompt"]`，但目前是 `""`
 - 沒有 `user_id`，所以這裡找不到
- 正確做法：
 1. `/generate-audio` 時，把 `user_id + prompt + task_id` 存 DB
 2. `/audio-callback` 用 callback 的 `task_id` 去 DB 撈回 `user_id` 和 `prompt`
 3. 再用這些資訊插入你原本的 `insert_record`，或回推給 Discord

如果你想，我可以幫你把 `save_task_mapping` / `get_task_mapping` 用 DynamoDB 的程式骨架寫出來，直接貼進你現有專案用。



```
from flask import Flask, request, jsonify,
send_from_directory
from dotenv import load_dotenv
import os, io, uuid, datetime
import requests
from PIL import Image
import awsgi
import boto3
from db import insert_record, get_history
import json

load_dotenv()
HuggingFace_Token =
os.getenv("HUGGINGFACE_TOKEN")
Suno_Token = os.getenv("SUNO_TOKEN")
S3_BUCKET_NAME = os.getenv("S3_BUCKET_NAME")
HuggingFace_URL = "https://router.huggingface.co/hf-
inference/models/stabilityai/stable-diffusion-xl-base-
1.0"
SUNO_URL =
"https://api.sunoapi.org/api/v1/generate"

app = Flask(__name__)

s3 = boto3.client("s3") # 建立 s3 client
```



```
@app.route("/generate-image", methods=["POST"])
def generate_image():
    try:
        data = request.json
        prompt = data.get("prompt", "沒有收到圖片
prompt")
        user_id = data.get("user_id", "unknown")

        if not prompt:
            return jsonify({"success": False, "error": "缺少
prompt"}), 400

        headers = {"Authorization": f"Bearer
{HuggingFace_Token}"}
        payload = {"inputs": prompt}

        response = requests.post(HuggingFace_URL,
headers=headers, json=payload, timeout=60)

        if response.status_code != 200:
            return jsonify({
                "success": False,
                "error": f"Hugging Face API 錯誤:
{response.status_code}",
                "details": response.text
            }, 500

        content_type = response.headers.get('Content-
Type', '')

        if 'application/json' in content_type:
            error_data = response.json()
            return jsonify({
                "success": False,
                "error": "Hugging Face 回傳錯誤",
                "details": error_data
            }, 500

        if 'image' not in content_type:
            return jsonify({
                "success": False,
                "error": f"非預期的回傳格式: {content_type}"
```

```
    }), 500

    # 嘗試解析圖片
    try:
        image =
Image.open(io.BytesIO(response.content))
    except Exception as e:
        return jsonify({
            "success": False,
            "error": f"圖片解析失敗: {str(e)}"
        }), 500

    # 存到 S3
    timestamp =
datetime.datetime.now().strftime("%Y%m%d%H%M%S
")
    filename = f"
{timestamp}_{uuid.uuid4().hex[:8]}.png"
    s3_key = f"Images/{filename}"

    try:
        s3.put_object(
            Bucket=S3_BUCKET_NAME,
            Key=s3_key,
            Body=response.content,    # 直接用原始 bytes
            ContentType="image/png"
        )
    except Exception as e:
        return jsonify({
            "success": False,
            "error": f"上傳 S3 失敗: {str(e)}"
        }), 500

    public_url=f"https://{S3_BUCKET_NAME}.s3.amazonaw
s.com/{s3_key}"
    local_url=f"s3://{S3_BUCKET_NAME}/{s3_key}"
    # 存進 DynamoDB
    try:
        result = insert_record(
            user_id=user_id,
            prompt=prompt,
            file_url=public_url,
```

```
        record_type="image",
        status="success"
    )
except Exception as e:
    return jsonify({
        "statusCode": 500,
        "body": json.dumps({
            "message": "✖ 插入失敗",
            "error": str(e)
        })
    })

# 提供可存取的 url
try:
    presigned_url = s3.generate_presigned_url(
        ClientMethod="get_object",
        Params={
            "Bucket": S3_BUCKET_NAME,
            "Key": s3_key
        },
        ExpiresIn=3600 # 有效時間秒數
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"產生下載網址失敗: {str(e)}"
    }, 500)

return jsonify({
    "success": True,
    "download_url": presigned_url, # 👉 這是 S3
    "reply": f"Flask 收到圖片 prompt: {prompt}"
})

except Exception as e:
    return jsonify({
        "success": False,
        "error": str(e)
    }, 500)

@app.route('/generate-audio', methods=['POST'])
```

```
def generate_audio():
    try:
        print("=== /generate-audio 被呼叫 ===")

        data = request.json
        print("收到的 JSON:", data)

        prompt = data.get('prompt')
        # discord_channel_id =
        data.get('discord_channel_id')
        # discord_user_id = data.get('discord_user_id')

        if not prompt:
            print("❌ 缺少 prompt")
            return jsonify({"success": False, "error": "缺少 prompt"}), 400

        payload = {
            "prompt": prompt,
            "style": "古典",
            "title": "AI Generated Music",
            "customMode": True,
            "instrumental": True,
            "model": "V3_5",
            "callBackUrl": "https://xpapysqd2i.execute-
api.us-east-1.amazonaws.com/prod/audio-callback"
        }

        print("Suno payload:", payload)

        headers = {
            "Authorization": f"Bearer {Suno_Token}",
            "Content-Type": "application/json"
        }

        print("正在呼叫 Suno API...")
        response = requests.post(SUNO_URL,
            json=payload, headers=headers, timeout=60)

        print("Suno 回應狀態:", response.status_code)
        print("Suno 回應內容:", response.text)

        if response.status_code != 200:
```

```
        return jsonify({
            "success": False,
            "error": f"Suno API 錯誤:
{response.status_code}",
            "details": response.text
        }), 500

    suno_resp = response.json()
    print("解析後 Suno JSON:", suno_resp)

    # ★ 抓 taskId (Suno 回傳格式有時不同)
    suno_request_id = (
        suno_resp.get("id")
        or suno_resp.get("requestId")
        or suno_resp.get("data", {}).get("taskId")
    )

    if not suno_request_id:
        print("✗ Suno 沒有回傳 taskId")
        return jsonify({
            "success": False,
            "error": "Suno 未回傳 taskId"
        }), 500

    # 回傳給 Discord bot，可以用來顯示“開始生成中”
    return jsonify({
        "success": True,
        "task_id": suno_request_id,
        "message": "音樂生成中"
    })

except Exception as e:
    print("✗ Flask 在 /generate-audio 發生錯誤:", e)
    return jsonify({"success": False, "error": str(e)}), 500

@app.route('/audio-callback', methods=['POST'])
def audio_callback():
    print("★ 進到 audio_callback")
    try:
        print("=== 收到 Suno callback ===")
        data = request.json
        suno_items = data["data"]["data"]
```

```
for item in suno_items:
    audio_url = item.get("source_audio_url")
    music_id = item["id"]

    if not audio_url:
        print("❌ 沒有 audio_url")
        return jsonify({
            "success": False,
            "error": "Suno 沒有回傳 audio_url"
        }), 500

    # 下載音檔
    mp3 = requests.get(audio_url).content

    # 上傳 S3
    timestamp =
datetime.datetime.now().strftime("%Y%m%d%H%M%S")
    filename = f"
{timestamp}_{uuid.uuid4().hex[:8]}.mp3"
    s3_key = f"Audios/{filename}"

    try:
        s3.put_object(
            Bucket=S3_BUCKET_NAME,
            Key=s3_key,
            Body=mp3,    # 直接用原始 bytes
            ContentType="audio/mpeg"
        )
    except Exception as e:
        return jsonify({
            "success": False,
            "error": f"上傳 S3 失敗: {str(e)}"
        }), 500

    public_url=f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"
    local_url=f"s3://{S3_BUCKET_NAME}/{s3_key}"

    # 存進 DynamoDB
    try:
        result = insert_record(
```

```
        user_id="no user id",
        prompt="no prompt",
        file_url=public_url,
        record_type="audio",
        status="success"
    )
except Exception as e:
    return jsonify({
        "statusCode": 500,
        "body": json.dumps({
            "message": "✖ 插入失敗",
            "error": str(e)
        })
    })
# 提供可存取的 url
try:
    presigned_url = s3.generate_presigned_url(
        ClientMethod="get_object",
        Params={
            "Bucket": S3_BUCKET_NAME,
            "Key": s3_key
        },
        ExpiresIn=3600 # 有效時間秒數
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"產生下載網址失敗: {str(e)}"
    }), 500

return jsonify({
    "success": True,
    "download_url": presigned_url, # 👉 這是 S3
    "reply": "Flask 收到音樂prompt"
})

except Exception as e:
    print("callback error:", e)
    return jsonify({"error": str(e)}), 500

@app.route("/history", methods=["GET"])
def history():
```

```

try:
    user_id = request.args.get("user_id") # 從 URL
    query 取得 user_id

    records = get_history(user_id)

    return jsonify({
        "success": True,
        "records": records # 形式 : [{ "prompt": "...",
    "file_url": "..."}], ...]
    })

except Exception as e:
    return jsonify({"success": False, "error": str(e)}), 500

if __name__ == '__main__':
    print("🚀 Flask 伺服器啟動中...")
    app.run(host='0.0.0.0', port=5000, debug=True)

我要把generate_audio的user_id跟prompt傳到audio-
callback儲存的user_id跟prompt

```

已思考 41s >

好，我直接幫你「把 `user_id` 跟 `prompt` 從 `/generate-audio` 傳到 `/audio-callback`，然後在 `callback` 用來存 DB」整套串起來。

我用 最簡單的方法：把資訊塞在 `callback URL` 的 `query string`

(API Gateway + awsgi → Flask 時，`?user_id=xxx&prompt=yyy` 會出現在 `request.args` 裡，可以直接拿)。

1 Flask：/generate-audio 加上 user_id + prompt 到 callbackUrl

先在檔案最上面多一個 `import`：

python

複製程式碼

```
from urllib.parse import quote_plus
```

然後修改 `/generate-audio`，重點是：

- 從 `body` 拿 `user_id`

- 把 `user_id` 、 `prompt` URL encode 之後，塞進 `callbackUrl` 的 query string

python

 複製程式碼

```

@app.route('/generate-audio', methods=['POST'])
def generate_audio():
    try:
        print("=== /generate-audio 被呼叫 ===")

        data = request.json
        print("收到的 JSON:", data)

        prompt = data.get('prompt')
        user_id = data.get('user_id', 'unknown') # 👉 新增: 從 body 拿 user_id

        if not prompt:
            print("❌ 缺少 prompt")
            return jsonify({"success": False, "error": "缺少 prompt"}), 400

        # 💡 安全編碼，避免空白、中文、特殊符號壞掉 URL
        encoded_prompt = quote_plus(prompt)
        encoded_user_id = quote_plus(str(user_id))

        base_callback = "https://xpapysqd2i.execute-api.us-east-1.amazonaws.com/p
        callback_url = f"{base_callback}?user_id={encoded_user_id}&prompt={encode

        payload = {
            "prompt": prompt,
            "style": "古典",
            "title": "AI Generated Music",
            "customMode": True,
            "instrumental": True,
            "model": "V3_5",
            "callbackUrl": callback_url # 👉 用含 user_id / prompt 的 callback
        }

        print("Suno payload:", payload)

        headers = {
            "Authorization": f"Bearer {Suno_Token}",
            "Content-Type": "application/json"
        }

```

```
print("正在呼叫 Suno API...")
response = requests.post(SUNO_URL, json=payload, headers=headers, timeout=10)

print("Suno 回應狀態:", response.status_code)
print("Suno 回應內容:", response.text)

if response.status_code != 200:
    return jsonify({
        "success": False,
        "error": f"Suno API 錯誤: {response.status_code}",
        "details": response.text
    }), 500

suno_resp = response.json()
print("解析後 Suno JSON:", suno_resp)

suno_request_id = (
    suno_resp.get("id")
    or suno_resp.get("requestId")
    or suno_resp.get("data", {}).get("taskId")
)

if not suno_request_id:
    print("✗ Suno 沒有回傳 taskId")
    return jsonify({
        "success": False,
        "error": "Suno 未回傳 taskId"
    }), 500

return jsonify({
    "success": True,
    "task_id": suno_request_id,
    "message": "音樂生成中"
})

except Exception as e:
    print("✗ Flask 在 /generate-audio 發生錯誤:", e)
    return jsonify({"success": False, "error": str(e)}), 500
```

2 Flask : /audio-callback 收到時，把 query string 取出來用

Suno callback 打回來時會叫你這個 URL：

text

複製程式碼

```
https://.../audio-callback?user_id=123456&prompt=%E4%BD%A0%E7%9A%84prompt
```

API Gateway + awsgi 會把這些 query string 變成 Flask 的 `request.args`。

你在 `/audio-callback` 開頭加上這兩行：

python

複製程式碼

```
@app.route('/audio-callback', methods=['POST'])
def audio_callback():
    print("★ 進到 audio_callback")
    try:
        print("=== 收到 Suno callback ===")

        # 📌 從 query string 拿回 user_id 和 prompt
        user_id = request.args.get("user_id", "no user id")
        prompt = request.args.get("prompt", "no prompt")

        print("callback user_id:", user_id)
        print("callback prompt:", prompt)

        data = request.json
        suno_items = data["data"]["data"]

        for item in suno_items:
            audio_url = item.get("source_audio_url")
            music_id = item["id"]

            if not audio_url:
                print("✗ 沒有 audio_url")
                return jsonify({
                    "success": False,
                    "error": "Suno 沒有回傳 audio_url"
                }), 500

            mp3 = requests.get(audio_url).content

            timestamp = datetime.datetime.now().strftime("%Y%m%d%H%M%S")
            filename = f"{timestamp}_{uuid.uuid4().hex[:8]}.mp3"
            s3_key = f"Audios/{filename}"
```

```
try:
    s3.put_object(
        Bucket=S3_BUCKET_NAME,
        Key=s3_key,
        Body=mp3,
        ContentType="audio/mpeg"
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"上傳 S3 失敗: {str(e)}"
    }), 500

public_url = f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"
local_url = f"s3://{S3_BUCKET_NAME}/{s3_key}"

# ♦ 這裡改成用真正的 user_id 和 prompt
try:
    result = insert_record(
        user_id=user_id,
        prompt=prompt,
        file_url=public_url,
        record_type="audio",
        status="success"
    )
except Exception as e:
    return jsonify({
        "statusCode": 500,
        "body": json.dumps({
            "message": "✖ 插入失敗",
            "error": str(e)
        })
    })

try:
    presigned_url = s3.generate_presigned_url(
        ClientMethod="get_object",
        Params={
            "Bucket": S3_BUCKET_NAME,
            "Key": s3_key
        },
        ExpiresIn=3600
    )
```

```

    )
    except Exception as e:
        return jsonify({
            "success": False,
            "error": f"產生下載網址失敗: {str(e)}"
        }), 500

    return jsonify({
        "success": True,
        "download_url": presigned_url,
        "reply": "Flask 收到音樂prompt"
    })

except Exception as e:
    print("callback error:", e)
    return jsonify({"error": str(e)}), 500

```

小提醒：你現在 `for item in suno_items:` 裡面 `return` 在迴圈裡，所以只會處理第一首，之後有空可以再改成處理完全部再回傳。

3 Discord Bot：/create_audio 要把 user_id 丟給 Flask

現在你的 `/create_audio` payload 只有 `prompt`：

python

複製程式碼

```

@bot.command()
async def create_audio(ctx, *, prompt: str="未輸入prompt"):
    try:
        payload = {"prompt": prompt}
        response = requests.post(f"{FLASK_API_URL}/generate-audio", json=payload,
        ...

```

要改成也把 Discord user id 送過去，讓 Flask 可以拿到：

python

複製程式碼

```

@bot.command()
async def create_audio(ctx, *, prompt: str="未輸入prompt"):
    try:
        payload = {
            "prompt": prompt,

```

```

        "user_id": str(ctx.author.id)  # 📌 帶使用者 ID
    }
    response = requests.post(f"{FLASK_API_URL}/generate-audio", json=payload,
                             data = response.json())

    if not data.get("success"):
        await ctx.send(f"❌ Flask 回傳錯誤: {data.get('error') or data}")
        return

    task_id = data.get("task_id")
    await ctx.send(f"✅ 已送出 Suno 任務, task_id: `{task_id}`, 完成後會自動存進")

except Exception as e:
    await ctx.send(f"❌ 呼叫 Flask 失敗: {e}")

```

4 總結一下資料流

1. Discord /create_audio

→ POST /generate-audio · body 有 prompt + user_id

2. /generate-audio

→ 把 prompt + user_id encode 進 callbackUrl 的 query string
→ 呼叫 Suno

3. Suno 生完音樂

→ POST 到 https://.../audio-callback?user_id=xxx&prompt=yyy

4. /audio-callback

→ request.args 拿回 user_id / prompt
→ 下載 mp3 → 上 S3 → insert_record(user_id, prompt, ...)

之後你 /history?user_id=某個 Discord ID 查出來的 audio 記錄，就會是對應那個使用者、那個 prompt 的音樂了 🎧

如果你之後想改成用 DynamoDB 以 task_id 做 mapping，而不是塞 query string，也可以再一起調整；但目前這個做法已經可以直接用，而且改動最少。



name 'quote_plus' is not defined

這個錯誤超單純，就是我剛剛叫你用 `quote_plus`，但還沒把它 `import` 進來 😂

你現在的錯誤：

text

複製程式碼

```
name 'quote_plus' is not defined
```

代表 Python 找不到這個函式。

1 在最上面補一行 import

在你檔案一堆 import 那一區，加這一行就好：

python

複製程式碼

```
from flask import Flask, request, jsonify, send_from_directory
from dotenv import load_dotenv
import os, io, uuid, datetime
import requests
from PIL import Image
import awsgi
import boto3
from db import insert_record, get_history
import json
from urllib.parse import quote_plus # 👉 加這一行
```

然後下面用的地方就可以直接用：

python

複製程式碼

```
encoded_prompt = quote_plus(prompt)
encoded_user_id = quote_plus(str(user_id))
```

2 如果你不想多一個 from-import (另一種寫法)

你也可以改成這樣 (擇一使用就好，不要兩個都加)：

python

複製程式碼

```
import urllib.parse

encoded_prompt = urllib.parse.quote_plus(prompt)
encoded_user_id = urllib.parse.quote_plus(str(user_id))
```

3 小檢查清單

1. 確認 只有一種寫法：

- 要嘛 `from urllib.parse import quote_plus + quote_plus(...)`
- 要嘛 `import urllib.parse + urllib.parse.quote_plus(...)`

2. 存檔、重新打包 / 部署 Lambda

3. 再用 Discord `/create_audio` 或 Postman 測一次 `/generate-audio`

如果加完 `import` 後，又出現新的錯誤（例如 Suno API 回錯、callback 又有問題），把新的錯誤訊息貼出來，我陪你一起解。

📄 🍌 🗨️ ⬆️ ↺️ ...

怎麼在lambda新增python套件

已思考 26s >

核心觀念：

Lambda 不是幫你 `pip install` 的環境，你要先把套件裝好，打包成 zip 上傳（或是做成 Layer 再掛上去）。

我用你現在這個專案當例子（Flask + awsgi + Pillow + dotenv），教你兩種常用方法：

方法一：跟程式一起打包（最直覺）

適合：一兩個 Lambda Function，程式 + 套件一起 zip 上傳。

1. 選好要裝的套件

你程式裡有用到的第三方套件大概是：

text

📄 複製程式碼


```
flask
awscli
python-dotenv
Pillow
requests
# boto3 不用裝，Lambda 內建
```

2. 在本機建立打包資料夾

在你專案資料夾（有 `lambda_function.py` 或 `app.py` 那個目錄）開終端機：

```
bash 複製程式碼

mkdir package
```

3. 用 pip 安裝到這個資料夾

⚠ 建議用 Python 版本跟 Lambda 一樣的環境來裝（例如 Lambda 是 python 3.11，你就用 3.11）。

```
bash 複製程式碼

pip install flask awscli python-dotenv Pillow requests -t package
```

結束後，`package/` 裡會有一堆套件資料夾。

4. 把你的程式放進去

例如你的檔案結構可以變成：

```
text 複製程式碼

project_root/
  package/
    flask/
    awscli/
    PIL/
    requests/
    ...
    lambda_function.py  ➡ 把這個放進來
    db.py               ➡ 你自訂的檔案也一起放
```

做法就是把 `lambda_function.py`、`db.py`、`app.py` 這些檔案 **移動或複製** 進 `package/` 裡。

5. 壓成 zip

在 `package` 的裡面全選壓縮 (zip 內容要是檔案本身，不要多一層 `package` 資料夾)：

```
bash
```

[複製程式碼](#)

```
cd package
zip -r ../lambda_deploy.zip .
```

這樣會在上一層得到 `lambda_deploy.zip`。

6. 上傳到 Lambda

1. 進 AWS Console → Lambda → 選你的 function
2. Code 來源那邊選：
 - 「Upload from」→ `.zip file`
3. 選剛剛的 `lambda_deploy.zip` 上傳
4. 儲存 / Deploy

這樣 Lambda 裡就會有：

- 你的 `lambda_function.py`
- 以及所有 `pip install` 的套件

小提醒：

- “`boto3` Lambda 內建，不用打包”
- “有些 C 編譯的套件 (比如 Pillow) 最好用 Linux 環境打包，不然 Windows 版的 `.pyd` 不能在 Lambda 跑。

真的有問題時可以用 WSL / Docker / Cloud9 來 `pip install` 再打包。”

方法二：做成 Lambda Layer (比較乾淨)

適合：多個 Lambda 要共用同一批套件 (例如 Flask/awsgi 等)。

1. 在本機準備 layer 資料夾

```
bash
```

[複製程式碼](#)

```
mkdir mylayer
cd mylayer
mkdir python
```

Lambda Layer 的規定是：

zip 裡面要有一個 `python/` 資料夾，套件裝在裡面。

2. 把套件裝到 python 裡

bash

 複製程式碼

```
pip install flask awsgi python-dotenv Pillow requests -t python
```

結束後資料結構：

text

 複製程式碼

```
mylayer/  
  python/  
    flask/  
    awsgi/  
    PIL/  
    requests/  
    ...
```

3. 壓成 zip

bash

 複製程式碼

```
zip -r mylayer.zip python
```

4. 在 AWS Console 建 Layer

1. AWS Console → Lambda → 左邊選「Layers」
2. Create layer
3. Name：例如 `flask-suno-layer`
4. 上傳 `mylayer.zip`
5. Runtime 勾你 Lambda 用的版本（例如 Python 3.11）
6. 建立 Layer

5. 把 Layer 掛到你的 Lambda function

1. 到你的 Lambda function 頁面
2. 「Configuration」→ 左邊選 Layers
3. Add a layer
4. 選「Custom layers」→ 選剛剛建立的那個 → 儲存

這樣你的 Lambda 裡就會多出這些套件可用，程式 zip 裡就只要放 `lambda_function.py`、`db.py` 等自己的檔案即可。

哪些套件不用你裝？

Lambda Python runtime 內建：

- `boto3`
- `botocore`
- 一些標準庫 (`json`, `os`, `datetime` ...)

你程式裡這幾個是自己要提供的：

- `flask`
- `awscli`
- `python-dotenv`
- `Pillow` (`PIL`)
- `requests`

快速建議 (針對你現在這個專案)

你現在是 **Flask + awscli + S3 + DynamoDB**，又只有一個 Lambda function 的樣子：

- 先用「方法一：跟程式一起打包」就好，比較好理解。
- 如果後面你拆成很多 Lambda，或發現 zip 一直重傳很煩，再把公共套件挪去 Layer。

如果你跟我說你現在 Lambda 的部署方式 (在本機 zip 上傳？用 Serverless framework？還是用 SAM？)，我可以直接幫你寫一個完整的小步驟腳本 (包含 pip 指令 + 資料夾結構)，照貼就能跑。



`from urllib.parse import quote_plus`
這要下載什麼套件

`from urllib.parse import quote_plus` 這個完全不用裝任何套件 🙌

- `urllib` 是 Python 內建標準庫
- `urllib.parse` 也是標準庫裡的一個 module
- `quote_plus` 就是 `urllib.parse` 裡的一個函式

只要你的 Python 是 3.x，寫這行就可以用了：

```
python
```

複製程式碼

```
from urllib.parse import quote_plus
```

在 Lambda 也一樣，因為 Lambda 的 Python runtime 本身就帶標準庫，你不用在 zip 裡額外放什麼 `urllib`、也不用 `pip install urllib`。

什麼情況才需要自己打包套件？

只有像這些才要自己裝 + 打包：

- `flask`
- `awscli`
- `python-dotenv`
- `Pillow` (`PIL`)
- `requests`
- 其他第三方套件 (`pip install` 來的)

像 `os` , `json` , `datetime` , `urllib.parse` 這種都屬於 **Python 內建標準庫**，不用裝。

所以你的狀況總結一下：

- 之前報 `name 'quote_plus' is not defined` 是因為少了 **import**，不是缺套件
- 現在只要在檔案上面加這行就好：

python

複製程式碼

```
from urllib.parse import quote_plus
```

就可以正常用，不用額外下載任何東西。

📄 🍌 🗨️ ⬆️ ↺️ ...

await ctx.send(f"✅ 從 Flask 取得檔案內容:\n{file_resp}")
讓bot等待帶三分鐘

你可以用 `asyncio.sleep()` 讓 bot 「停三分鐘再送訊息」。

1 先在檔案上面 import

在你的 Discord bot 檔案最上面加：

```
python
```

[複製程式碼](#)

```
import asyncio
```

2 在指令裡面加三分鐘等待

你現在原本是這樣：

```
python
```

[複製程式碼](#)

```
file_resp = data.get("download_url")
await ctx.send(f"✅ 從 Flask 取得檔案內容:\n{file_resp}")
```

如果你想要 先等三分鐘再送這個訊息，改成：

```
python
```

[複製程式碼](#)

```
file_resp = data.get("download_url")

# 先回一個「處理中」的訊息（可選）
await ctx.send("🎵 正在從 Flask 取得檔案內容，可能需要一點時間...")

# 等三分鐘 (180 秒)
await asyncio.sleep(180)

# 再送真正內容
await ctx.send(f"✅ 從 Flask 取得檔案內容:\n{file_resp}")
```

3 小提醒

- `asyncio.sleep(180)` 只能在 `async def` 裡用（你現在指令 function 已經是 `async` 的，OK）。
- 這只是「固定等三分鐘」，不會真的確認 Suno 什麼時候生好；之後你如果想做「真的等 callback 完成再回覆」，可以改成：
 - `/generate-audio` 只回 `task_id`
 - callback 存 DB
 - Discord 另外有 `/history` 讓使用者自己查，或用 webhook 主動推通知

不過先用 `sleep(180)` 當作簡單版流程是沒問題的。



`import asyncio`
要安裝甚麼套件

`asyncio` 跟剛剛的 `urllib.parse` 一樣，是 **Python 內建標準庫**，完全不用安裝任何套件
👍

所以：

python

複製程式碼

```
import asyncio
```

這行直接寫就可以用了，在：

- 你本機的 Python
- Lambda (Python 3.x runtime)
- 任何正常的 Python 3.x 環境

都不需要 `pip install asyncio` 。

什麼時候才需要安裝套件？

只有像這些才需要 `pip install` + 打包上 Lambda：

- `flask`
- `awscli`
- `python-dotenv`
- `Pillow`
- `requests`
- 其他第三方套件

像 `asyncio`，`os`，`json`，`datetime`，`urllib` 這些都是 Python 官方內建的，直接 `import` 就好，不用安裝。



```
from flask import Flask, request, jsonify,
send_from_directory
from dotenv import load_dotenv
import os, io, uuid, datetime
import requests
from PIL import Image
import awsgi
import boto3
from db import insert_record, get_history
import json
from urllib.parse import quote_plus

load_dotenv()
HuggingFace_Token =
os.getenv("HUGGINGFACE_TOKEN")
Suno_Token = os.getenv("SUNO_TOKEN")
S3_BUCKET_NAME = os.getenv("S3_BUCKET_NAME")
HuggingFace_URL = "https://router.huggingface.co/hf-
inference/models/stabilityai/stable-diffusion-xl-base-
1.0"
SUNO_URL =
"https://api.sunoapi.org/api/v1/generate"

app = Flask(__name__)

s3 = boto3.client("s3") # 建立 s3 client

@app.route("/generate-image", methods=["POST"])
def generate_image():
    try:
        data = request.json
        prompt = data.get("prompt", "沒有收到圖片
prompt")
        user_id = data.get("user_id", "unknown")

        if not prompt:
            return jsonify({"success": False, "error": "缺少
prompt"}), 400

        headers = {"Authorization": f"Bearer
{HuggingFace_Token}"}
```



```
payload = {"inputs": prompt}

response = requests.post(HuggingFace_URL,
headers=headers, json=payload, timeout=60)

if response.status_code != 200:
    return jsonify({
        "success": False,
        "error": f"Hugging Face API 錯誤:
{response.status_code}",
        "details": response.text
    }), 500

content_type = response.headers.get('Content-
Type', '')

if 'application/json' in content_type:
    error_data = response.json()
    return jsonify({
        "success": False,
        "error": "Hugging Face 回傳錯誤",
        "details": error_data
    }), 500

if 'image' not in content_type:
    return jsonify({
        "success": False,
        "error": f"非預期的回傳格式: {content_type}"
    }), 500

# 嘗試解析圖片
try:
    image =
Image.open(io.BytesIO(response.content))
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"圖片解析失敗: {str(e)}"
    }), 500

# 存到 S3
timestamp =
datetime.datetime.now().strftime("%Y%m%d%H%M%S
```

```
)
    filename = f"
{timestamp}_{uuid.uuid4().hex[:8]}.png"
    s3_key = f"Images/{filename}"

    try:
        s3.put_object(
            Bucket=S3_BUCKET_NAME,
            Key=s3_key,
            Body=response.content,    # 直接用原始 bytes
            ContentType="image/png"
        )
    except Exception as e:
        return jsonify({
            "success": False,
            "error": f"上傳 S3 失敗: {str(e)}"
        }), 500

public_url=f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"
local_url=f"s3://{S3_BUCKET_NAME}/{s3_key}"
# 存進 DynamoDB
try:
    result = insert_record(
        user_id=user_id,
        prompt=prompt,
        file_url=public_url,
        record_type="image",
        status="success"
    )
except Exception as e:
    return jsonify({
        "statusCode": 500,
        "body": json.dumps({
            "message": "❌ 插入失敗",
            "error": str(e)
        })
    })

# 提供可存取的 url
try:
    presigned_url = s3.generate_presigned_url(
```

```
ClientMethod="get_object",
Params={
    "Bucket": S3_BUCKET_NAME,
    "Key": s3_key
},
ExpiresIn=3600 # 有效時間秒數
)
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"產生下載網址失敗: {str(e)}"
    }), 500

return jsonify({
    "success": True,
    "download_url": presigned_url, # 👉 這是 S3
presigned URL
    "reply": f"Flask 收到圖片prompt: {prompt}"
})

except Exception as e:
    return jsonify({
        "success": False,
        "error": str(e)
    }), 500

@app.route('/generate-audio', methods=['POST'])
def generate_audio():
    try:
        print("=== /generate-audio 被呼叫 ===")

        data = request.json
        print("收到的 JSON:", data)

        user_id = data.get('user_id', 'unknown')
        prompt = data.get('prompt')

        encoded_prompt = quote_plus(prompt)
        encoded_user_id = quote_plus(str(user_id))

        if not prompt:
            print("❌ 缺少 prompt")
```

```
        return jsonify({"success": False, "error": "缺少  
prompt"}), 400

    base_callback = "https://xpapysqd2i.execute-  
api.us-east-1.amazonaws.com/prod/audio-callback"  
    callback_url = f"{base_callback}?user_id=  
{encoded_user_id}&prompt={encoded_prompt}"  
    payload = {  
        "prompt": prompt,  
        "style": "古典",  
        "title": "AI Generated Music",  
        "customMode": True,  
        "instrumental": True,  
        "model": "V3_5",  
        "callBackUrl": callback_url  
    }

    print("Suno payload:", payload)

    headers = {  
        "Authorization": f"Bearer {Suno_Token}",  
        "Content-Type": "application/json"  
    }

    print("正在呼叫 Suno API...")  
    response = requests.post(SUNO_URL,  
                             json=payload, headers=headers, timeout=60)

    print("Suno 回應狀態:", response.status_code)  
    print("Suno 回應內容:", response.text)

    if response.status_code != 200:  
        return jsonify({  
            "success": False,  
            "error": f"Suno API 錯誤:  
{response.status_code}",  
            "details": response.text  
        }), 500

    suno_resp = response.json()  
    print("解析後 Suno JSON:", suno_resp)

    # ★ 抓 taskId (Suno 回傳格式有時不同)
```

```
suno_request_id = (  
    suno_resp.get("id")  
    or suno_resp.get("requestId")  
    or suno_resp.get("data", {}).get("taskId")  
)
```

```
if not suno_request_id:  
    print("❌ Suno 沒有回傳 taskId")  
    return jsonify(  
        "success": False,  
        "error": "Suno 未回傳 taskId"  
    ), 500
```

```
# 回傳給 Discord bot，可以用來顯示“開始生成中”  
return jsonify(  
    "success": True,  
    "task_id": suno_request_id,  
    "message": "音樂生成中"  
)
```

```
except Exception as e:  
    print("❌ Flask 在 /generate-audio 發生錯誤:", e)  
    return jsonify({"success": False, "error": str(e)}), 500
```

```
@app.route('/audio-callback', methods=['POST'])  
def audio_callback():  
    print("★ 進到 audio_callback")  
    try:  
        print("=== 收到 Suno callback ===")  
        data = request.json  
        suno_items = data["data"]["data"]  
  
        for item in suno_items:  
            audio_url = item.get("source_audio_url")  
            music_id = item["id"]  
  
            if not audio_url:  
                print("❌ 沒有 audio_url")  
                return jsonify(  
                    "success": False,  
                    "error": "Suno 沒有回傳 audio_url"  
                ), 500
```

```
# 下載音檔
mp3 = requests.get(audio_url).content

# 上傳 S3
timestamp =
datetime.datetime.now().strftime("%Y%m%d%H%M%S")
filename = f"
{timestamp}_{uuid.uuid4().hex[:8]}.mp3"
s3_key = f"Audios/{filename}"

try:
    s3.put_object(
        Bucket=S3_BUCKET_NAME,
        Key=s3_key,
        Body=mp3,    # 直接用原始 bytes
        ContentType="audio/mpeg"
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"上傳 S3 失敗: {str(e)}"
    }), 500

user_id = request.args.get("user_id", "no user id")
prompt = request.args.get("prompt", "no
prompt")

public_url=f"https://{S3_BUCKET_NAME}.s3.amazonaw
s.com/{s3_key}"
local_url=f"s3://{S3_BUCKET_NAME}/{s3_key}"

# 存進 DynamoDB
try:
    result = insert_record(
        user_id=user_id,
        prompt=prompt,
        file_url=public_url,
        record_type="audio",
        status="success"
    )
except Exception as e:
```

```
        return jsonify({
            "statusCode": 500,
            "body": json.dumps({
                "message": "✖ 插入失敗",
                "error": str(e)
            })
        })
# 提供可存取的 url
try:
    presigned_url = s3.generate_presigned_url(
        ClientMethod="get_object",
        Params={
            "Bucket": S3_BUCKET_NAME,
            "Key": s3_key
        },
        ExpiresIn=3600 # 有效時間秒數
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"產生下載網址失敗: {str(e)}"
    }), 500

    return jsonify({
        "success": True,
        "download_url": presigned_url, # 👉 這是 S3
presigned URL
        "reply": "Flask 收到音樂prompt"
    })

except Exception as e:
    print("callback error:", e)
    return jsonify({"error": str(e)}), 500

@app.route("/history", methods=["GET"])
def history():
    try:
        user_id = request.args.get("user_id") # 從 URL
query 取得 user_id

        records = get_history(user_id)

        return jsonify({
```

```

        "success": True,
        "records": records # 形式 : [{ "prompt": "...",
        "file_url": "..."}], ...]
    })

except Exception as e:
    return jsonify({"success": False, "error": str(e)}), 500

if __name__ == '__main__':
    print("🚀 Flask 伺服器啟動中...")
    app.run(host='0.0.0.0', port=5000, debug=True)
    讓generate-audio收callbackurl回傳的"download_url"

```

已思考 50s >

先直接講重點：

在你現在這種「Suno 用 callback 打回來、Flask 部署在 Lambda + API Gateway」的架構裡，`/generate-audio` 沒辦法「直接收到」`/audio-callback` 回傳的 `download_url`。

原因不是你程式少寫一行，而是「HTTP + Lambda 的機制」本來就做不到這種同步串接，特別是你還經過 Suno callback 這一段。

我分三塊講：

一、為什麼 `generate-audio` 收不到 callback 的 `download_url` ？

目前實際流程長這樣：

1. Discord `/create_audio`

→ 呼叫 API Gateway → Lambda → awscli → Flask `/generate-audio`

2. `/generate-audio`

→ 呼叫 Suno `/api/v1/generate`，給它 `callbackUrl=https://.../audio-callback?user_id=...&prompt=...`

→ 很快回一個 `task_id` 給你（你現在就是這樣做）

3. 過一段時間之後（幾秒到幾十秒）

Suno 從它的伺服器主動 `POST` 到你的 `https://.../audio-callback?...`

→ API Gateway → Lambda → awscli → Flask `/audio-callback`

→ 你在這裡下載 mp3，上傳 S3，回傳 `{"success": True, "download_url": ...}`

關鍵點：

- `/generate-audio` 和 `/audio-callback` 是兩個完全獨立的 HTTP 請求。
- `audio-callback` 的回應是回給 Suno 的，不會「傳回」到當初呼叫 Suno 的那個 `/generate-audio`。
- 你的 Lambda 又在 API Gateway 後面，最多只能跑 29–30 秒，不可能在那裡等 Suno 幾十秒甚至幾分鐘。

所以：

`/generate-audio` 想「等 Suno callback 完成後再回傳 `download_url`」在 API Gateway + Lambda 這個環境基本上是不可能的
(就算你硬 `time.sleep(180)`，API Gateway 也會先 timeout 掉)

二、正確的做法：用「共享儲存」把兩個 request 串起來

既然不能直接把 callback 的 response 傳給原本的 request，那就只能走這種模式：

1. `/generate-audio` 呼叫 Suno 拿到 `task_id` 後，先在 DB 存一筆任務資料：

python

 複製程式碼

```
# 假想的函式，實作可以用 DynamoDB / RDS / 你自己的 db.py
save_audio_task(
    task_id=suno_request_id,
    user_id=user_id,
    prompt=prompt,
    status="pending",
    file_url=None
)
```

2. Suno callback 打到 `/audio-callback` 的時候，你：

- 從 JSON 拿 `task_id = data["data"]["task_id"]`
- 上傳 S3 拿 `public_url`
- 更新那一筆 task：把 `status` 改成 `done`，`file_url` 寫進去

python

 複製程式碼

```
update_audio_task(
    task_id=task_id,
    status="done",
    file_url=public_url
)
```

3. Discord 端要拿 `download_url` 有兩種做法：

A. 使用者自己查 (你現在已經有 `/history` , 其實很好用)

- `/create_audio` 只回「已送出, task_id: XXX」
- 等個幾秒~幾十秒, 使用者 `/history`
→ 你的 `get_history(user_id)` 去 DB 撈該 user 的所有記錄 (包括剛剛那首的 `file_url`)

B. callback 主動推給 Discord (比較爽但要多寫一點)

在 `/audio-callback` 裡:

- 你已經有 `user_id` (從 query string)
- S3 上傳完拿到 `public_url`
- 這時可以直接呼叫 Discord Webhook 或 Bot token 去 `POST` 一個訊息:

python

複製程式碼

```
# 假設你有 DISCORD_WEBHOOK_URL
requests.post(DISCORD_WEBHOOK_URL, json={
    "content": f"@{user_id}> 你的音樂好了: {public_url}"
})
```

這樣當音樂生成完成, 機器人就會「主動貼訊息」到指定頻道,
而不是靠 `/generate-audio` 卡在那邊等。

三、那「讓 `/generate-audio` 收 `download_url`」有沒有任何變通?

在 Lambda + API Gateway 上: 幾乎不行

就算你:

- 在 `/generate-audio` 呼叫 Suno 之後
- 進入一個迴圈, 每 3 秒去查 DB 看 callback 寫進來的 `file_url` 了沒

像這樣:

python

複製程式碼

```
import time

timeout = 25 # API Gateway 最大 29s 左右
interval = 3
elapsed = 0

while elapsed < timeout:
```

```
task = get_audio_task(suno_request_id)
if task and task.get("file_url"):
    # 代表 callback 已經跑完、S3 url 存好了
    return jsonify({
        "success": True,
        "download_url": task["file_url"],
        "task_id": suno_request_id
    })
time.sleep(interval)
elapsed += interval

# 超過時間就先回 task_id
return jsonify({
    "success": True,
    "task_id": suno_request_id,
    "message": "音樂還在生成中，稍後請用 /history 查詢"
})
```

這種「偽同步」也最多只能等 20 幾秒，超過就 timeout。

Suno 生成音樂通常不可能穩定在 10 秒內完成，所以實戰上很不穩。

在自己主機 / EC2 / Docker 上跑 Flask 就沒這個限制

如果你不是放 Lambda，而是自己跑一台 Flask server：

- 就可以真的 `while` loop 等 1-2 分鐘都沒人管
- 但 callback 依然 **不是直接把 HTTP response 傳回前一個 request**，你還是要透過 DB 這種「共享狀態」才知道某個 task 何時完成。

四、結論幫你整理一下

你現在這句話：

「讓 `generate-audio` 收 `callbackurl` 回傳的 `download_url`」

在 **Suno callback + Lambda + API Gateway** 這個架構下，**不能用「直接傳」的方式做到**。

正確方向是：

1. 在 `/generate-audio` 拿到 `task_id` 後，先把這個任務 (`task_id + user_id + prompt`) 存到 DB。
2. 在 `/audio-callback` 裡用 `task_id` 找回這筆任務，生完音樂、上傳 S3 後，把 `file_url` 和狀態寫回 DB。
3. Discord 端：

- 要嘛：
 - `/create_audio` 回傳 `task_id` + 「請稍後用 `/history` 查詢」
- 要嘛：
 - `/audio-callback` 那邊用 Webhook 主動通知 Discord，貼出 `download_url`

如果你願意，我可以下一步直接幫你：

- 設計一個簡單的 `audio_tasks` DynamoDB table schema
- 寫出 `save_audio_task` / `update_audio_task` / `get_audio_task` 的 Python 函式骨架
- 幫你把它嵌進你貼的那隻 Flask 程式裡，變成一個「實際可跑」的版本。

< 1/2 >      ...

audio_callback的return會傳給generate_audio的 response嗎

不會。完全不會。

`audio_callback` 的 `return` 跟 `generate_audio` 的 `response` 是兩條完全不同的 HTTP 流程。

你可以想成這樣的時間線 📌：

1. Discord `/create_audio`
→ 打到 API Gateway
→ Lambda → Flask `/generate-audio`
2. `/generate-audio` 做的事：
 - 呼叫 Suno 的 `/generate` API
 - Suno 說：「好，我開始做，這是你的 `task_id`」
 - `/generate-audio` 把這個 `task_id` 回給 Discord
👉 到這裡，`/generate-audio` 的 `response` 就結束了
3. 過一段時間後（幾秒、幾十秒）：
Suno 生完音樂，自己發一個 新的 HTTP 請求：

text

 複製程式碼

Suno → POST `https://你的網址/audio-callback?user_id=...&prompt=...`

這時才會進到 Flask 的 `/audio-callback`：

python

 複製程式碼

```
@app.route('/audio-callback', methods=['POST'])
def audio_callback():
    ...
    return jsonify({"success": True, "download_url": ...})
```

4. 這個 `return` 是回給 **Suno** 的 HTTP 請求，
不是回到當初 Discord 打來的那個 `/generate-audio`。

用一句話總結：

`generate_audio` 處理的是「Discord → 你」這個請求；
`audio_callback` 處理的是「Suno → 你」這個**另一個**請求。
 兩個請求彼此獨立，所以 `audio_callback` 的回傳 **不可能** 自動變成 `generate_audio` 的 response。

要把兩邊串起來，只能靠**共同的資料來源**（例如 DB、Redis、檔案、記憶體服務等），
而不是指望「callback 的 return 自動傳回前一個 request」。



```
from flask import Flask, request, jsonify,
send_from_directory
from dotenv import load_dotenv
import os, io, uuid, datetime
import requests
from PIL import Image
import awsgi
import boto3
from db import insert_record, get_history
import json
from urllib.parse import quote_plus

load_dotenv()
HuggingFace_Token =
os.getenv("HUGGINGFACE_TOKEN")
Suno_Token = os.getenv("SUNO_TOKEN")
S3_BUCKET_NAME = os.getenv("S3_BUCKET_NAME")
HuggingFace_URL = "https://router.huggingface.co/hf-
inference/models/stabilityai/stable-diffusion-xl-base-
```

```
1.0"
SUNO_URL =
"https://api.sunoapi.org/api/v1/generate"

app = Flask(__name__)
s3 = boto3.client("s3") # 建立 s3 client

@app.route("/generate-image", methods=["POST"])
def generate_image():
    try:
        data = request.json
        prompt = data.get("prompt", "沒有收到圖片
prompt")
        user_id = data.get("user_id", "unknown")

        if not prompt:
            return jsonify({"success": False, "error": "缺少
prompt"}), 400

        headers = {"Authorization": f"Bearer
{HuggingFace_Token}"}
        payload = {"inputs": prompt}

        response = requests.post(HuggingFace_URL,
headers=headers, json=payload, timeout=60)

        if response.status_code != 200:
            return jsonify({
                "success": False,
                "error": f"Hugging Face API 錯誤:
{response.status_code}",
                "details": response.text
            }, 500

        content_type = response.headers.get('Content-
Type', '')

        if 'application/json' in content_type:
            error_data = response.json()
            return jsonify({
                "success": False,
                "error": "Hugging Face 回傳錯誤",
                "details": error_data
```

```
    }), 500

    if 'image' not in content_type:
        return jsonify({
            "success": False,
            "error": f"非預期的回傳格式: {content_type}"
        }), 500

    # 嘗試解析圖片
    try:
        image =
        Image.open(io.BytesIO(response.content))
    except Exception as e:
        return jsonify({
            "success": False,
            "error": f"圖片解析失敗: {str(e)}"
        }), 500

    # 存到 S3
    timestamp =
    datetime.datetime.now().strftime("%Y%m%d%H%M%S")
    filename = f"
    {timestamp}_{uuid.uuid4().hex[:8]}.png"
    s3_key = f"Images/{filename}"

    try:
        s3.put_object(
            Bucket=S3_BUCKET_NAME,
            Key=s3_key,
            Body=response.content,    # 直接用原始 bytes
            ContentType="image/png"
        )
    except Exception as e:
        return jsonify({
            "success": False,
            "error": f"上傳 S3 失敗: {str(e)}"
        }), 500

    public_url=f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"
    local_url=f"s3://{S3_BUCKET_NAME}/{s3_key}"
```

```
# 存進 DynamoDB
try:
    result = insert_record(
        user_id=user_id,
        prompt=prompt,
        file_url=public_url,
        record_type="image",
        status="success"
    )
except Exception as e:
    return jsonify({
        "statusCode": 500,
        "body": json.dumps({
            "message": "❌ 插入失敗",
            "error": str(e)
        })
    })

# 提供可存取的 url
try:
    presigned_url = s3.generate_presigned_url(
        ClientMethod="get_object",
        Params={
            "Bucket": S3_BUCKET_NAME,
            "Key": s3_key
        },
        ExpiresIn=3600 # 有效時間秒數
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"產生下載網址失敗: {str(e)}"
    }), 500

return jsonify({
    "success": True,
    "download_url": presigned_url, # 👉 這是 S3
    "reply": f"Flask 收到圖片 prompt: {prompt}"
})

except Exception as e:
```



```
        return jsonify({
            "success": False,
            "error": str(e)
        }), 500

@app.route('/generate-audio', methods=['POST'])
def generate_audio():
    try:
        print("=== /generate-audio 被呼叫 ===")

        data = request.json
        print("收到的 JSON:", data)

        user_id = data.get('user_id', 'unknown')
        prompt = data.get('prompt')

        encoded_prompt = quote_plus(prompt)
        encoded_user_id = quote_plus(str(user_id))

        if not prompt:
            print("❌ 缺少 prompt")
            return jsonify({"success": False, "error": "缺少 prompt"}), 400

        base_callback = "https://xpapysqd2i.execute-api.us-east-1.amazonaws.com/prod/audio-callback"
        callback_url = f"{base_callback}?user_id={encoded_user_id}&prompt={encoded_prompt}"
        payload = {
            "prompt": prompt,
            "style": "古典",
            "title": "AI Generated Music",
            "customMode": True,
            "instrumental": True,
            "model": "V3_5",
            "callbackUrl": callback_url
        }

        print("Suno payload:", payload)

        headers = {
            "Authorization": f"Bearer {Suno_Token}",
            "Content-Type": "application/json"
```

```
}

print("正在呼叫 Suno API...")
response = requests.post(SUNO_URL,
json=payload, headers=headers, timeout=60)

print("Suno 回應狀態:", response.status_code)
print("Suno 回應內容:", response.text)

if response.status_code != 200:
    return jsonify({
        "success": False,
        "error": f"Suno API 錯誤:
{response.status_code}",
        "details": response.text
    }), 500

suno_resp = response.json()
print("解析後 Suno JSON:", suno_resp)

# ★ 抓 taskId ( Suno 回傳格式有時不同 )
suno_request_id = (
    suno_resp.get("id")
    or suno_resp.get("requestId")
    or suno_resp.get("data", {}).get("taskId")
)

if not suno_request_id:
    print("✗ Suno 沒有回傳 taskId")
    return jsonify({
        "success": False,
        "error": "Suno 未回傳 taskId"
    }), 500

# 回傳網址
timestamp =
datetime.datetime.now().strftime("%Y%m%d%H%M%S")
filename = f"
{timestamp}_{uuid.uuid4().hex[:8]}.mp3"
s3_key = f"Audios/{filename}"
try:
    presigned_url = s3.generate_presigned_url(
```

```
ClientMethod="get_object",
Params={
    "Bucket": S3_BUCKET_NAME,
    "Key": s3_key
},
ExpiresIn=3600 # 有效時間秒數
)
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"產生下載網址失敗: {str(e)}"
    }), 500

# 回傳給 Discord bot，可以用來顯示“開始生成中”
return jsonify({
    "download_url": presigned_url,
    "success": True,
    "task_id": suno_request_id,
    "message": "音樂生成中"
})

except Exception as e:
    print("✖ Flask 在 /generate-audio 發生錯誤:", e)
    return jsonify({"success": False, "error": str(e)}), 500

@app.route('/audio-callback', methods=['POST'])
def audio_callback():
    print("★ 進到 audio_callback")
    try:
        print("=== 收到 Suno callback ===")
        data = request.json
        suno_items = data["data"]["data"]

        for item in suno_items:
            audio_url = item.get("source_audio_url")
            music_id = item["id"]

            if not audio_url:
                print("✖ 沒有 audio_url")
                return jsonify({
                    "success": False,
                    "error": "Suno 沒有回傳 audio_url"
                }), 500
```

```
# 下載音檔
mp3 = requests.get(audio_url).content

# 上傳 S3
timestamp =
datetime.datetime.now().strftime("%Y%m%d%H%M%S")
filename = f"
{timestamp}_{uuid.uuid4().hex[:8]}.mp3"
s3_key = f"Audios/{filename}"

try:
    s3.put_object(
        Bucket=S3_BUCKET_NAME,
        Key=s3_key,
        Body=mp3,    # 直接用原始 bytes
        ContentType="audio/mpeg"
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"上傳 S3 失敗: {str(e)}"
    }), 500

user_id = request.args.get("user_id", "no user id")
prompt = request.args.get("prompt", "no
prompt")

public_url=f"https://{S3_BUCKET_NAME}.s3.amazonaw
s.com/{s3_key}"
local_url=f"s3://{S3_BUCKET_NAME}/{s3_key}"

# 存進 DynamoDB
try:
    result = insert_record(
        user_id=user_id,
        prompt=prompt,
        file_url=public_url,
        record_type="audio",
        status="success"
    )
```

```
except Exception as e:
    return jsonify({
        "statusCode": 500,
        "body": json.dumps({
            "message": "✖ 插入失敗",
            "error": str(e)
        })
    })
# 提供可存取的 url
try:
    presigned_url = s3.generate_presigned_url(
        ClientMethod="get_object",
        Params={
            "Bucket": S3_BUCKET_NAME,
            "Key": s3_key
        },
        ExpiresIn=3600 # 有效時間秒數
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"產生下載網址失敗: {str(e)}"
    }), 500

return jsonify({
    "success": True,
    "download_url": presigned_url, # 👉 這是 S3
    "reply": "Flask 收到音樂prompt"
})

except Exception as e:
    print("callback error:", e)
    return jsonify({"error": str(e)}), 500

@app.route("/history", methods=["GET"])
def history():
    try:
        user_id = request.args.get("user_id") # 從 URL
        query 取得 user_id

        records = get_history(user_id)
```



```
<Message>The specified key does not exist.
</Message>
<Key>Audios/20251128163235_4e50cc0f.mp3</Key>
<RequestId>DYYPYBNK8RCJMDCP</RequestId>
<HostId>mNN7z1PE85MVuIZsitcq2/LbKNwSjSOuPs9v
61Nz4J8rKNq0TkIMMr566MdkPSWFrS0a9YqM3FFKdw
p2g656Lpmonxh/7Bjq</HostId>
</Error>
```

已思考 33s >

你現在看到的這個錯誤，其實是「你先幫一個根本不存在的 S3 檔案做了預簽網址」，所以 S3 很誠實地回你：NoSuchKey 🤪

一、先拆解一下現在發生了什麼

1. 在 `/generate-audio` 裡

你做了這段（重點）：

python

複製程式碼

```
# ★ 這裡只是產生一個「名字」，沒有真的上傳！
timestamp = datetime.datetime.now().strftime("%Y%m%d%H%M%S")
filename = f"{timestamp}_{uuid.uuid4().hex[:8]}.mp3"
s3_key = f"Audios/{filename}"

presigned_url = s3.generate_presigned_url(
    ClientMethod="get_object",
    Params={
        "Bucket": S3_BUCKET_NAME,
        "Key": s3_key
    },
    ExpiresIn=3600
)
```

也就是說：

- 你先隨便取了一個檔名：`Audios/20251128163235_4e50cc0f.mp3`
- 然後幫這個「尚未存在的物件」產生一組 **可讀取 URL**

👉 但是 你沒有在這個 function 裡呼叫 `put_object` 上傳任何東西，所以 S3 裡其實根本沒有這個 key。

2. 真正有上傳檔案的是 `/audio-callback`

在 `audio_callback` 裡才真的有上傳：

python

複製程式碼

```
timestamp = datetime.datetime.now().strftime("%Y%m%d%H%M%S")
filename = f"{timestamp}_{uuid.uuid4().hex[:8]}.mp3"
s3_key = f"Audios/{filename}"

s3.put_object(
    Bucket=S3_BUCKET_NAME,
    Key=s3_key,
    Body=mp3,
    ContentType="audio/mpeg"
)
```

注意幾件事：

1. 這裡又重新生成了一個 `timestamp` + `uuid` → 跟 `/generate-audio` 裡的不一樣
2. 所以這裡實際上傳的 key 完全是另一個，例如：
`Audios/20251128164010_a1b2c3d4.mp3`
3. 所以你在 `/generate-audio` 裡產生的這個 URL：

text

複製程式碼

`https://testusage1124.s3.amazonaws.com/Audios/20251128163235_4e50cc0f.mp3?...`

指向的是一個從來沒被上傳過的 key，當然會得到：

xml

複製程式碼

```
<Code>NoSuchKey</Code>
<Message>The specified key does not exist.</Message>
```

二、要怎麼修？兩條路選一條

路線 A：乾脆 不要在 `/generate-audio` 回傳 `download_url`

這是最乾淨、也是架構最合理的做法。

因為流程本來就是：

1. `/generate-audio` → 只是丟任務給 Suno，拿 `task_id`
2. `/audio-callback` → Suno 生完音樂打回來，你再上傳 S3，這時才有真正的檔案網址

所以 `/generate-audio` 比較合理的回傳是：

python

複製程式碼

```
return jsonify({
    "success": True,
    "task_id": suno_request_id,
    "message": "音樂生成中"
})
```

然後你可以：

- 讓使用者晚點用 `/history` 查
- 或在 `audio_callback` 裡主動用 Discord webhook/機器人發訊息把 `download_url` 貼到頻道

這條路簡單、安全，而且符合「非同步任務」的設計。

路線 B：預先決定一個 `s3_key`，讓 callback 用同一個 key

如果你真的想讓 `/generate-audio` 先給一個「將來會生效」的 URL，可以這樣做：

- 1 在 `/generate-audio` 先決定好 `s3_key`，並帶到 callback

把你現在 `/generate-audio` 的 S3 產生段移到 Suno 前面，然後 callback URL 裡多傳一個 `s3_key`：

python

複製程式碼

```
@app.route('/generate-audio', methods=['POST'])
def generate_audio():
    ...
    user_id = data.get('user_id', 'unknown')
    prompt = data.get('prompt')

    if not prompt:
```

```
...

# 先決定這次要用的 S3 key (只產生名字，不上傳)
timestamp = datetime.datetime.now().strftime("%Y%m%d%H%M%S")
filename = f"{timestamp}_{uuid.uuid4().hex[:8]}.mp3"
s3_key = f"Audios/{filename}"

# URL encode
encoded_prompt = quote_plus(prompt)
encoded_user_id = quote_plus(str(user_id))
encoded_s3_key = quote_plus(s3_key)

base_callback = "https://xpapysqd2i.execute-api.us-east-1.amazonaws.com/prod/"
callback_url = (
    f"{base_callback}"
    f"?user_id={encoded_user_id}"
    f"&prompt={encoded_prompt}"
    f"&s3_key={encoded_s3_key}"
)

payload = {
    "prompt": prompt,
    "style": "古典",
    "title": "AI Generated Music",
    "customMode": True,
    "instrumental": True,
    "model": "V3_5",
    "callbackUrl": callback_url
}

...

# Suno 回 200 & 有 task_id 後，再用「同一個 s3_key」做預簽網址
presigned_url = s3.generate_presigned_url(
    ClientMethod="get_object",
    Params={
        "Bucket": S3_BUCKET_NAME,
        "Key": s3_key
    },
    ExpiresIn=3600
)

return jsonify({
```

```

    "success": True,
    "task_id": suno_request_id,
    "download_url": presigned_url, # ⚠ 一開始點可能還會 NoSuchKey (因為檔案還沒生成)
    "message": "音樂生成中"
})

```

這樣 `/generate-audio` 回給你的 URL，指向的是「未來 `audio_callback` 會上傳 mp3 的位置」。

2 在 `/audio-callback` 用同一個 `s3_key` 上傳檔案

把你現在 `callback` 裡「再生一個新檔名」的地方改掉，改成讀 `query string` 裡的

`s3_key`：

python

複製程式碼

```

@app.route('/audio-callback', methods=['POST'])
def audio_callback():
    print("★ 進到 audio_callback")
    try:
        print("=== 收到 Suno callback ===")
        data = request.json
        suno_items = data["data"]["data"]

        # 從 query string 拿 user_id、prompt、s3_key
        user_id = request.args.get("user_id", "no user id")
        prompt = request.args.get("prompt", "no prompt")
        s3_key = request.args.get("s3_key") # ★ 關鍵

        if not s3_key:
            # 萬一沒帶，就 fallback 用新檔名 (避免整個壞掉)
            timestamp = datetime.datetime.now().strftime("%Y%m%d%H%M%S")
            filename = f"{timestamp}_{uuid.uuid4().hex[:8]}.mp3"
            s3_key = f"Audios/{filename}"

        for item in suno_items:
            audio_url = item.get("source_audio_url")
            music_id = item["id"]

            if not audio_url:
                ...

```

```
mp3 = requests.get(audio_url).content

# ★ 這裡直接用從 generate-audio 傳來的 s3_key
try:
    s3.put_object(
        Bucket=S3_BUCKET_NAME,
        Key=s3_key,
        Body=mp3,
        ContentType="audio/mpeg"
    )
except Exception as e:
    ...

public_url=f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"

try:
    result = insert_record(
        user_id=user_id,
        prompt=prompt,
        file_url=public_url,
        record_type="audio",
        status="success"
    )
except Exception as e:
    ...

# 如果你要在 callback 再產生一個 presigned url 也可以
presigned_url = s3.generate_presigned_url(
    ClientMethod="get_object",
    Params={
        "Bucket": S3_BUCKET_NAME,
        "Key": s3_key
    },
    ExpiresIn=3600
)

return jsonify({
    "success": True,
    "download_url": presigned_url,
    "reply": "Flask 收到音樂prompt"
})
```

```
except Exception as e:  
    ...
```

這樣：

- `/generate-audio` 先決定名字 → 給你一個預簽網址（此時檔案還沒上去）
- Suno 生完音樂，呼叫 `callback`，把檔案上傳到 **同一個** `s3_key`
- 過幾秒後，你再點 `/generate-audio` 回的那個網址，就會變成有東西可讀

目前你 `NoSuchKey` 的根本原因，就是 `generate-audio` 和 `audio-callback` 用的是兩個完全不同的 `s3_key`。

三、小總結（用一句話講）

- 現在的網址會 `NoSuchKey`，是因為：
 - `/generate-audio` 先幫一個「從來沒被上傳的 key」做了預簽 URL。
- 真正有上傳 mp3 的地方，是 `/audio-callback`，而且那邊用了完全不同的檔名。
- 修法：
 - 最乾淨：`/generate-audio` 不要回 `download_url`，只回 `task_id`，檔案網址由 `audio-callback` 寫 DB 或發 Discord。
 - 或：在 `/generate-audio` 先決定好 `s3_key`，用 query string 傳給 `audio-callback`，讓它用同一個 `s3_key` 上傳。

你如果跟我說你想走「只回 `task_id`」那條路，還是「預先給 URL」那條路，我可以幫你吧整個 Flask + Discord 那邊的 code 串到完全一致版本給你用。



bot收到指令後，先回傳，請等待

已思考 15s >

可以，這個超簡單：在指令一開始先 `await ctx.send("請等待")`，後面照原本流程跑就好。

也可以順便把後面的結果再補一個新訊息。

我直接用你現在的兩個指令改一版給你參考。

1 /create_photo : 先回「已收到，請等待」

python

複製程式碼

```

@bot.command()
async def create_photo(ctx, *, prompt: str = "未輸入prompt"):
    try:
        # ✅ 先回一個「收到指令」的訊息
        await ctx.send(f"✅ 已收到圖片指令，請稍候產生圖片中...")

        payload = {"prompt": prompt, "user_id": str(ctx.author.id)}
        response = requests.post(f"{FLASK_API_URL}/generate-image", json=payload,
                                data = response.json())

        if not data.get("success"):
            await ctx.send(f"❌ Flask 回傳錯誤: {data.get('error', '未知錯誤')}")
            return

        file_url = data.get("download_url")
        if not file_url:
            await ctx.send("❌ Flask 沒有回傳檔案 URL")
            return

        file_resp = requests.get(file_url)
        if file_resp.status_code == 200:
            with io.BytesIO(file_resp.content) as f:
                await ctx.send(file=discord.File(f, filename="generated.png"))
        else:
            await ctx.send("❌ 下載圖片失敗")

    except Exception as e:
        await ctx.send(f"❌ 呼叫 Flask 失敗: {e}")

```

2 /create_audio : 先回「請等待」，再回網址 / 狀態

假設你現在的 /generate-audio 會回 success 、 task_id 、 (可能) download_url :

python

複製程式碼

```

@bot.command()
async def create_audio(ctx, *, prompt: str = "未輸入prompt"):

```

```

try:
    # ✅ 先回一個「請等待」
    await ctx.send("🎵 已收到音樂指令，請稍候，我正在請 Flask / Suno 生成音樂...")

    payload = {
        "prompt": prompt,
        "user_id": str(ctx.author.id),
    }
    response = requests.post(f"{FLASK_API_URL}/generate-audio",
                             json=payload, timeout=30)
    data = response.json()

    if not data.get("success"):
        await ctx.send(f"❌ Flask 回傳錯誤: {data.get('error', '未知錯誤')}")
        return

    task_id = data.get("task_id")
    download_url = data.get("download_url") # 如果你現在有回這個欄位

    # 這裡看你的後端設計：
    if download_url:
        await ctx.send(f"✅ 音樂生成中，但網址已經準備好：\n{download_url}")
    else:
        await ctx.send(f"✅ 已送出 Suno 任務，task_id: `{task_id}`\n請稍後使用")

except Exception as e:
    await ctx.send(f"❌ 呼叫 Flask 失敗: {e}")

```

3 如果你想「只有一則訊息」，可以用 edit

進階一點的寫法是：

```

python
msg = await ctx.send("🎵 已收到音樂指令，請稍候產生中...")

# 跑完 Flask / Suno 之後
await msg.edit(content=f"✅ 音樂生成中，task_id: `{task_id}`")

```

複製程式碼

這樣就變成一則訊息「先顯示請等待，後來更新成結果」，比較乾淨。

總結：

在任何 `@bot.command()` 裡，一開始加一行 `await ctx.send("請等待 / 已收到指令...")` 就可以達到你要的「bot 收到指令後先回傳「請等待」」的效果，之後的處理流程不需要改動太多。

